Google 288
Algorithm Primer

Read this in three days, before you learn the code solutions

---

43 patterns · 288 questions · 738 follow-ups

Every algorithm used by the easy, medium and hard Google study path, explained once, with a short template and the traps to avoid.

How to use it. Each pattern page says how to spot it, the idea in plain words, a short template with clear names, the cost, the traps, and the follow-ups to expect. At the bottom of each pattern is the list of study-path questions that use it, with one line telling you what the algorithm does there. Read a pattern, close the book, and write the template from memory. Then the long solutions in the app will read as “this is pattern 24 with a twist” instead of a wall of code.

Contents

The three day plan 4

Python toolbox and complexity 4

Follow-up families that repeat everywhere 5

Pattern finder: from the words of the question to the section 5

Day 1: Counting, scanning and searching 7

10. Dict of per-key state (update, expire, rank) . . . 7
13. Sets and counters across two collections . . . 7
34. Group by a canonical key . . . 8
30. Count pairs by looking up each partner . . . 8
8. Runs and groups: extend while it fits, restart when it breaks . . . 9
15. Prefix sums, tier 1: precompute once, query in O(1) . . . 9
33. Prefix sums, tier 2: running total plus a dict of earlier totals . . . 10
20. Two pointers moving toward each other (or along two lists) . . . 10
40. Sliding window: grow on the right, shrink on the left . . . 11
12. Greedy: scan once, carry the best so far, act only when forced . . . 11
11. Binary search on a yes/no condition . . . 12
14. bisect on a sorted list: position, range count, keep sorted . . . 12
16. Sorted intervals: merge while they touch, then read gaps or search . . . 13
2. Sweep over events in time order . . . 13

Day 2: Strings, design and trees 15

21. Walk two strings at once with two indices . . . 15
43. Split into pieces, fix each piece, join (and word wrap) . . . 15
24. Digits and characters: peel one at a time and carry . . . 16
42. Build or transform a grid with index arithmetic . . . 16
22. Simulate a machine: a class with state and one branch per event . . . 17
18. A dict for lookup plus an ordered container for who is next . . . 17
41. Parent pointers: store who is above each node and climb . . . 18
37. Random pick without repeats: swap with the last and pop . . . 18
27. Nested structures: a stack of contexts, or recursion with a cache . . . 19
25. Look at the input once and keep only a small summary . . . 19
3. Tree recursion, tier 1: visit a node, ask children, report upward . . . 20
36. Tree recursion, tier 2: pass a rule down, return a node or yes/no up . . . 20
31. Tree DP: build a parent order once, add up from the leaves . . . 21
38. Build and serialize: write out a structure, read it back with the same walk . . . 21
17. Trie: nested dicts, one character per level . . . 22

Day 3: Recursion, dynamic programming, heaps and graphs 23

7. Backtracking: choose, recurse, undo . . . 23
39. Games: win or lose with a cache, and guessing with feedback . . . 23
19. Rolling state: the next state comes only from the previous one . . . 24
5. Recurrence on a position or a range (memo or table) . . . 24
26. Grid DP: each cell from its neighbours above and to the left . . . 25

32. Counting or reaching an amount: the knapsack loop . . . 25
4. A heap of what is available: pop the best, put back the changed one . . . 26
35. K-way merge: a heap of the next item from each source . . . 27
23. Top-K by frequency: Counter plus nsmallest . . . 27
1. Frontier search (BFS or DFS), tier 1: spread, count, measure distance . . . 27
28. Frontier search, tier 2: free steps, early exit, neighbours worked out on the fly . . . 28
29. Topological sort (Kahn) and cycle detection . . . 29
9. Union-find: merge the pairs, then ask who is together . . . 29
6. Best-first search with a heap: Dijkstra and its minimax twin . . . 30

Appendix: algorithms hiding inside single questions **31**
Fenwick tree (binary indexed tree) . . . 31
Segment tree (first slot that fits, range minimum, range updates) . . . 31
Rolling hash (Rabin-Karp) for repeated substrings . . . 31
Sparse table (range minimum in $O(1)$) . . . 31
Huffman coding . . . 32
Fast doubling and matrix power (huge n recurrences) . . . 32
Group testing, augmenting paths and other named ideas . . . 32

The three day plan

Budget: 5 to 6 hours a day. Read a pattern for 10 minutes, then write its template from memory for 5 minutes. Do not copy; re-read only what you could not rebuild. Anything you cannot rebuild goes on a list for the end of the day.

Day 1: Counting, scanning and searching. Hash maps, prefix sums, two pointers, windows, greedy, binary search, sorted intervals and sweeps. Patterns 2, 8, 10, 11, 12, 13, 14, 15, 16, 20, 30, 33, 34, 40.

By the end of the day you can: Explain in one sentence why a dict lookup replaces an inner loop. Write a prefix sum, a sliding window and a binary search on an answer from memory. Merge intervals and run a sweep over events without looking.

Day 2: Strings, design and trees. Strings and digits, grids, simulations and caches, parent pointers, random structures, nested parsing, streaming, trees, serialisation and tries. Patterns 3, 17, 18, 21, 22, 24, 25, 27, 31, 36, 37, 38, 41, 42, 43.

By the end of the day you can: Write an LRU cache, a trie and a swap-and-pop random set from memory. State when a tree function returns a value and when it updates a global answer. Describe the rerooting formula and a length-prefixed encoding.

Day 3: Recursion, dynamic programming, heaps and graphs. Backtracking, games, dynamic programming, heaps, top-k, BFS and DFS, topological sort, union-find and Dijkstra. Patterns 1, 4, 5, 6, 7, 9, 19, 23, 26, 28, 29, 32, 35, 39.

By the end of the day you can: Write the choose, recurse, undo skeleton and a memoised state function. Write BFS, Kahn, union-find and Dijkstra with a heap from memory. For any new problem, name the state of a DP or the frontier of a search before coding.

Day 4 (optional): open the app, pick one question per pattern, and write the main solution without notes. If you stall, the stall is the pattern you need to reread, not the question.

Python toolbox and complexity

Pick the target complexity from the input size. Interview inputs are chosen so one family of algorithms fits.

n up to	Target	Typical tools
20 to 25	$O(2^n)$, $O(n!)$	backtracking, bitmask DP
1,000	$O(n^2)$	table DP, all pairs, nested loops
100,000	$O(n \log n)$	sort, heap, binary search, balanced structure
1,000,000 and up	$O(n)$ or $O(\log n)$	one pass, hashing, two pointers, math

Costs of the tools you will use all day.

Structure	Fast	Slow (avoid in loops)
`list`	append, pop, index $O(1)$	`insert(0)`, `pop(0)`, `x in list`: $O(n)$
`dict`, `set`	get, set, delete, in: $O(1)$ average	iteration order is insertion order only
`deque`	append, popleft, appendleft $O(1)$	indexing in the middle $O(n)$
`heapq`	push, pop $O(\log n)$; peek `h[0]` $O(1)$	search or delete a middle item $O(n)$
sorted list + `bisect`	search $O(\log n)$	`insort` $O(n)$ (shifts items)
`str`	index, slice $O(k)$	`s += t` in a loop: build a list and join

Standard library you are expected to know.

- `Counter(items)`: counts; `most_common(k)`; subtract with `c[x] -= 1`. `defaultdict(list)` or `defaultdict(int)` removes the "key exists?" check.
- `deque`: BFS queue, sliding window, 0-1 BFS. `OrderedDict.move_to_end` and `popitem(last=False)` give an LRU in a few lines.
- `heapq`: `heappush`, `heappop`, `heapify`, `heapreplace`, `nsmallest(k, xs, key=...)`. It is a min-heap: push `-x` for a max-heap. Push tuples `(priority, tiebreak, item)` so items never get compared.
- `bisect_left(a, x)` first index with `a[i] >= x`; `bisect_right(a, x)` first index with `a[i] > x`; `insort`.
- `lru_cache(maxsize=None)` from functools memoises a function of hashable arguments (use tuples, not lists). From itertools: `combinations`, `permutations`, `accumulate` (prefix sums). `zip(*matrix)` transposes.
- `sorted(xs, key=lambda x: (-count[x], x))`: sort by several criteria; sort is stable. Bits: `x & (x-1)` clears the lowest set bit, `1 << i` is bit i, `x >> i & 1` reads it.
- Recursion depth is about 1000 in Python: for deep trees or chains use an explicit stack or a queue (BFS), or raise the recursion limit and say why it is safe.

The loop to run on every question. (1) Restate it and ask about duplicates, ties, empty input, negative numbers, the size of the input. (2) Say a brute force and its cost. (3) Name the pattern in this book and the state or data structure it needs. (4) State the complexity before coding. (5) Code with clear names. (6) Test on an empty case, a one item case, a tie, and a case that breaks your first idea. (7) Be ready for the follow-ups listed with each pattern.

Follow-up families that repeat everywhere

Across the 288 questions the same follow-ups return in different clothes. Learn one answer to each family and you can answer most of them.

The input is a stream or does not fit in memory. Keep a small summary instead of the data: a running max, a size-k heap, a count dict, a set of ids from the smaller file. For two huge inputs partition both by a hash of the key and process one partition at a time. Many machines: map (emit key and value), shuffle by key, reduce.

Make it thread safe. One lock around each method is correct and simple; finer locks (per key or per bucket) give more concurrency but need a lock order to avoid deadlock. Read-mostly data can use copy-on-write. Say what is atomic.

Items can also be deleted or updated. A plain sorted list or prefix sum breaks: move to a Fenwick tree or a segment tree, a sorted container with $O(\log n)$ updates, lazy deletion in a heap, or swap-and-pop in a list with a position dict. Union-find cannot split groups: process offline in reverse or rebuild.

Return the path or the items, not only the number. Store a parent or choice for each state when you first improve it, then walk back from the goal: `parent[v] = u` in BFS and Dijkstra, the chosen split in DP, the chosen index in a greedy scan.

Weights, costs or priorities are added. BFS becomes Dijkstra (non-negative weights, or 0-1 BFS for 0 and 1), greedy may stop being correct (re-check the exchange argument), plain counts become sums stored in the same tables.

Recursion is too deep or too slow. Convert to an explicit stack or a queue; add a cache to turn exponential into polynomial; compute subtree values by sweeping a parent-first order backwards.

Do it in place, or in $O(1)$ extra space. Reuse the input as the table (swap each value into its own slot, encode old and new state in different bits), or use two pointers instead of a set after sorting.

Online arrivals, many queries, repeated calls. Precompute once (prefix sums, adjacency, a trie, a sorted list) and make each query cheap; for arrivals that cannot be sorted ahead use a heap or a balanced structure.

The input is huge numerically. Use modular arithmetic, matrix exponentiation or fast doubling for recurrences, a closed form for counting, or strings for numbers beyond machine size.

Prove or justify. Greedy: name the exchange argument and a counter-example for the neighbouring problem. Binary search: show the predicate is monotone. DP: state the meaning of the state. Dijkstra: weights are non-negative so a popped node is final.

Pattern finder: from the words of the question to the section

Read the question, match the words, open the section. Numbers are the section numbers used in the app.

#	Pattern	Words that give it away
1	Frontier search (BFS or DFS), tier 1	number of islands, flood fill, rotten oranges, shortest path in an unweighted graph, distance to nearest X.
2	Sweep over events in time order	minimum rooms, maximum overlap, skyline, who is online at time t, servers needed.
3	Tree recursion, tier 1	height, diameter, right side view, level order, delete leaves, evaluate expression tree.
4	A heap of what is available	minimum rooms, streaming median, reusable locker, schedule on identical CPUs, no two adjacent the same.
5	Recurrence on a position or a range (memo or table)	edit distance, longest palindromic subsequence, word break, split into expressions, count decompositions.
6	Best-first search with a heap	shortest path with weights, earliest arrival with timetables, minimise the maximum height on a path, swim in rising water.
7	Backtracking	all permutations / combinations, generate parentheses, count paths of length k, can this hand win, assign people to tasks.
8	Runs and groups	longest contiguous run, group consecutive items, streak, one replacement allowed.
9	Union-find	connected components, groups, are A and B related, add land one cell at a time, points within a radius.
10	Dict of per-key state (update, expire, rank)	remember the last time we saw X, top users, within the last 10 seconds, design a logger / counter.

#	Pattern	Words that give it away
11	Binary search on a yes/no condition	minimum speed / capacity / time such that ..., largest length such that ..., first bad version, median of two sorted arrays.
12	Greedy	minimum number of ..., maximum profit, largest number you can form, cover all holes, jumps.
13	Sets and counters across two collections	appears in both, not in the other list, duplicates, pair each x with 2x.
14	bisect on a sorted list	count in a range, k-th largest as items stream in, nearest value, longest increasing subsequence.
15	Prefix sums, tier 1	sum of a range, balanced split, pivot index, equal halves, area up to a height.
16	Sorted intervals	merge intervals, do bookings conflict, free time, insert an interval, union of two lists.
17	Trie	autocomplete, words with a prefix, longest match tokenizer, URL router, dictionary substrings.
18	A dict for lookup plus an ordered container for who is next	LRU cache, waitlist, round robin scheduler, remove the cheapest, undo / redo.
19	Rolling state	climbing stairs, house robber, paint houses, fibonacci, dice with limits.
20	Two pointers moving toward each other (or along two lists)	palindrome, reverse, sorted array and a target pair, merge two sorted lists, triplets that sum to zero.
21	Walk two strings at once with two indices	is a subsequence, lexicographic order in a custom alphabet, can board A become board B, similar sentences.
22	Simulate a machine	design tic-tac-toe, elevator system, iterator over lists, game of life, scorekeeper.
23	Top-K by frequency	k most frequent words, top chatters, task scheduler with cooldown, bottom k.
24	Digits and characters	add two big numbers, palindrome number, monotone digits, caesar cipher, k-th permutation.
25	Look at the input once and keep only a small summary	file does not fit in memory, stream, minimise the number of oracle calls, two logs.
26	Grid DP	unique paths, largest square, longest run of one colour, paths with checkpoints.
27	Nested structures	decode string 3[a2[c]], evaluate expression, nested list weight, expand placeholders.
28	Frontier search, tier 2	edge direction flips cost 1, reach with blocked nodes, shortest directed cycle through v, radios within a radius.
29	Topological sort (Kahn) and cycle detection	course schedule, build order, are these race results consistent, validate dependencies.
30	Count pairs by looking up each partner	pairs with difference k, two ways to write as a sum, rectangle / square from points.
31	Tree DP	total size of a folder, sum of distances from every node, time to inform everyone, cheapest cut.
32	Counting or reaching an amount	coin change, partition equal subset sum, target sum with plus and minus, unbounded items that fill a capacity.
33	Prefix sums, tier 2	number of subarrays with sum k, multiple of k, maximum subarray, equal endpoints.
34	Group by a canonical key	anagrams, same shape, shifted words, group strings that are equivalent.
35	K-way merge	merge k sorted lists, k smallest pair sums, sort a nearly sorted array, top-k subset sums.
36	Tree recursion, tier 2	valid BST, lowest common ancestor, in-order successor, mode in a tree.
37	Random pick without repeats	random set with O(1) delete, shuffle, unique random generator, no repeat within k songs.
38	Build and serialize	serialize / deserialize, construct tree from preorder and inorder, print a directory tree, encode a list of strings.
39	Games	who wins with optimal play, merge stacks game, guess the secret word, Wordle strategy.
40	Sliding window	longest/shortest subarray or substring such that ..., at most k distinct, minimum window, budget of changes.
41	Parent pointers	lowest common ancestor with parent links, validate a tree, employee hierarchy, is this a root.
42	Build or transform a grid with index arithmetic	rotate a matrix, range sum in a 2D matrix, draw a diamond, fill in snake order, block minimum.
43	Split into pieces, fix each piece, join (and word wrap)	snake_case to camelCase, format a number, justify text, wrap at width.

Day 1: Counting, scanning and searching

10. Dict of per-key state (update, expire, rank)

Hash maps and sets 8 questions: 1 easy, 7 medium, 0 hard

Spot it: "remember the last time we saw X", "top users", "within the last 10 seconds", "design a logger / counter".

Idea: Keep one dictionary whose key is the thing you track (message, user, word) and whose value is the small state about it: a timestamp, a count, a set, a deque. Every operation is a lookup, an update, or an expiry check on that one entry, so each call costs O(1) on average. To rank, keep a second structure (buckets by count, or sort at read time) instead of re-sorting on every update.

```
last_shown = {}
def should_print(now, message, window=10):
    if message in last_shown and now - last_shown[message] < window:
        return False
    last_shown[message] = now
    return True

hits = deque()                      # sliding window of hit times
def hit(now):  hits.append(now)
def count(now, window=300):
    while hits and hits[0] <= now - window:
        hits.popleft()              # expire from the old end
    return len(hits)
```

Cost: O(1) per call; expiry is amortised O(1) because each item is removed once. **Traps:** Decide first whether a rejected call refreshes the timestamp (usually it does not). Timestamps can repeat. Use < or <= consistently at the window edge.

Follow-ups: Out of order events (keep sorted, or a heap). Bounded memory (evict expired keys lazily or in a background sweep). Thread safety (one lock, or a lock per key). Millions of hits per second (bucket by second instead of storing every hit).

Solve these first: warm-up E, Robot Status Message Deduplication Window; then stretch M, Hit Counter.

Questions in this section (E easy, M medium, H hard):

- **E** **Robot Status Message Deduplication Window**: the pattern template above: show a message only if it was not shown in the last 10 seconds
- **M** **Active Communication Top Users**: a set of seen pairs, a Counter of partners per user, and buckets by count to read the top users in O(1)
- **M** **Filter Repeated Robot Status Messages**: the same rule applied to a stream of (timestamp, message) pairs; keep what is accepted
- **M** **Next-Word Predictor**: defaultdict(Counter): counts[a][b] += 1 for neighbouring words; remember the best next word per word
- **M** **Batch and Streaming Log Deduplication**: dict message -> first (or latest) timestamp depending on the policy; sort the result at the end
- **M** **Design a logger that returns top-K messages**: dict message -> list of timestamps; top_k counts each list inside the time range with bisect, then takes the k largest
- **M** **Website Activity Analytics: Unique Users + Session Time**: dict activity -> set of users for unique counts; dict user -> sorted events to add up login-to-logout session time
- **M** **Hit Counter**: one deque of hit times; drop the ones older than 300 seconds; the answer is the length

13. Sets and counters across two collections

Hash maps and sets 8 questions: 4 easy, 3 medium, 1 hard

Spot it: "appears in both", "not in the other list", "duplicates", "pair each x with 2x".

Idea: Put the first collection into a set (membership) or a Counter (multiplicity), then stream the second collection past it and test each item in O(1). A set answers "is it there"; a Counter answers "how many are left", and you decrement it as you use items up. When the smaller side fits in memory, always load the smaller side.

```
seen = set()
for value in first_list:
    if value in seen:
        return True                 # duplicate found
    seen.add(value)

remaining = Counter(values)         # pair each x with 2x
for x in sorted(remaining, key=abs):
    if remaining[x] > remaining[2 * x]:
        return []
    remaining[2 * x] -= remaining[x]
```

Cost: O(n + m) time, O(min(n, m)) extra space. **Traps:** Zero is its own double. Negative numbers need sorting by absolute value. Do not mutate the Counter while iterating its keys unless you iterate a sorted copy.

Follow-ups: Second input is a huge stream (load only the smaller set). No extra memory allowed (sort both and use two pointers, or use the array itself as the set by swapping). Three or more collections (intersect step by step).

Solve these first: warm-up E, Exclude First-K-Unique From Another List; then stretch H, Smallest Missing Positive Integer in Linear Time and Constant Extra ....

Questions in this section (E easy, M medium, H hard):

E **Exclude First-K-Unique From Another List**: the set holds the first k distinct values of list 1; keep list 2's values that are not in it
E **Positional Matches Between Two Lists**: no set needed: zip both lists and keep a where a == b
E **Contains Duplicate**: add values to a set as you go; the first value already in the set means a duplicate
E **Find the Duplicates**: values that appear in both lists, each reported once (set intersection, or two pointers on sorted lists)
M **Reconstruct original array from doubled shuffle**: Counter of the values; go from the smallest, pairing each x with 2x and using both counts up
M **Min deletions to avoid overlap in first k**: Counter of the left part and of the right part; for each value in both, delete min(left count, right count)
M **Filter Common Elements Per Window (Two Lists)**: build a set per window of the first list; keep the matching window of the second list minus that set
H **Smallest Missing Positive Integer in Linear Time and Constant ...**: use the array itself as the set: swap each value x into slot x - 1, then the first slot with the wrong value is the ...

34. Group by a canonical key

Hash maps and sets 5 questions: 1 easy, 3 medium, 1 hard

Spot it: "anagrams", "same shape", "shifted words", "group strings that are equivalent".

Idea: Compute a key that is identical for every member of one group and different across groups, then put items in a dict of lists under that key. The art is choosing the key: sorted letters or a 26-slot count tuple for anagrams, the differences between neighbouring letters (mod 26) for shifted words, the keyboard row for typeable words.

```
groups = defaultdict(list)
for word in words:
    key = tuple(sorted(word))              # or a 26-count tuple
    groups[key].append(word)
return list(groups.values())

def shift_key(word):                       # shifted-word key
    return tuple((ord(b) - ord(a)) % 26 for a, b in zip(word, word[1:]))
```

Cost: O(total letters * log L) with sorted keys, O(total letters) with count tuples. **Traps:** Lists are not hashable: convert the key to a tuple or string. The count-tuple key avoids the log factor. Decide how to treat case, spaces and Unicode.

Follow-ups: Different keyboard or alphabet. Subsets of letters instead of full anagrams. Words spread over many machines (hash the key to pick the machine).

Solve these first: warm-up E, Given an array of strings, return the words that can be typed using ...; then stretch H, Given two strings and two allowed operations, swapping positions of ....

Questions in this section (E easy, M medium, H hard):

E **Given an array of strings, return the words that can be typed ...**: key = the keyboard row of each letter; a word qualifies when its letters map to one row
M **Check if all substrings are anagrams of words**: key = the tuple of letter counts; keep the keys of the allowed words in a set and test every substring of length 3 or ...
M **Group anagrams**: key = the sorted letters (or a tuple of 26 letter counts)
M **Given an array of strings, return groups of "buddy strings," ...**: key = the differences between neighbouring letters, modulo 26 (a shifted word has the same differences)
H **Given two strings and two allowed operations, swapping ...**: same letters used, and the sorted list of letter frequencies must match: compare set(s) == set(t) and the sorted ...

30. Count pairs by looking up each partner

Hash maps and sets 5 questions: 0 easy, 4 medium, 1 hard

Spot it: "pairs with difference k", "two ways to write as a sum", "rectangle / square from points".

Idea: For every item, the partner you need is determined by the rule (v + k, target - v, the opposite corner). Store what you have seen in a Counter or set, then look the partner up in O(1) instead of scanning all other items. When a pair would be found twice, either count only one direction or divide the total by 2.

```
counts = Counter(values)
pairs = 0
for v in counts:
    if k == 0:
        pairs += counts[v] >= 2          # the partner is itself
    elif v + k in counts:
        pairs += 1

points = set(map(tuple, grid_points))     # squares from two opposite corners
for (x1, y1), (x2, y2) in combinations(points, 2):
    if x1 != x2 and y1 != y2 and abs(x1 - x2) == abs(y1 - y2):
        total += (x1, y2) in points and (x2, y1) in points
total //= 2                               # each square is seen from both diagonals
```

Cost: O(n) for single partner lookups, O(n^2) for all pairs of points. **Traps:** The partner may equal the item itself (k = 0). Count distinct pairs, not index pairs. Bucket pairs by a signature (sum of coordinates, squared length) to find rectangles without checking all quadruples.

Follow-ups: Stage 2 with new rules (keep the code adaptable). Duplicates allowed (count with multiplicity). Beat O(n^2) pairs (bucket by signature).

Solve these first: warm-up M, Progressive Array Partitioning; then stretch H, Maximum-Area Rectangle from Planar Points.

Questions in this section (E easy, M medium, H hard):

- M **Progressive Array Partitioning**: Counter of the values; stage 1 asks whether all the counts are different: len(set(counts.values())) == len(counts)
- M **K-diff Pairs in an Array**: the pattern template above: partner = v + k; for k == 0 count values that appear twice
- M **Count Axis-Aligned Squares Among Grid Points**: a set of points; two opposite corners fix the other two, so look both up; every square is found twice, divide by 2
- M **Sum of Two Squares in Two Ways**: enumerate every pair (a, b) with a <= b once, add a*a + b*b to a Counter, answer = sums counted exactly twice
- H **Maximum-Area Rectangle from Planar Points**: bucket every pair of points by (sum of coordinates, squared length); two pairs in one bucket are the diagonals of a ...

8. Runs and groups: extend while it fits, restart when it breaks

Arrays and prefix sums 9 questions: 2 easy, 7 medium, 0 hard

Spot it: "longest contiguous run", "group consecutive items", "streak", "one replacement allowed".

Idea: Walk the array once with a running length (or running sum). If item i fits with item i-1, extend the run; otherwise close it, record the best, and restart at 1. For "one change allowed", precompute for each index the run length ending there (left[i]) and starting there (right[i]); changing the element at i can join left[i-1] and right[i+1].

```
best = run = 1
for i in range(1, len(a)):
    run = run + 1 if a[i] >= a[i - 1] else 1
    best = max(best, run)

left = [1] * n;  right = [1] * n          # one change allowed
for i in range(1, n):
    if a[i] >= a[i - 1]: left[i] = left[i - 1] + 1
for i in range(n - 2, -1, -1):
    if a[i] <= a[i + 1]: right[i] = right[i + 1] + 1
best = max(left[i - 1] + 1 + right[i + 1] for i in range(1, n - 1)
           if a[i + 1] - a[i - 1] >= 0)
```

Cost: O(n) time, O(1) extra for one pass, O(n) for the left/right tables. **Traps:** Strict versus non-strict comparison. Remember to record the last run after the loop. A run of one element is valid. The joined run needs the two outer neighbours to be compatible.

Follow-ups: Return where the longest run is (store its start). Up to k modifications (then it is a sliding window). Many (l, r) queries (prefix arrays). Group by key per user.

Solve these first: warm-up E, First Out-of-Order Character; then stretch M, Count Super Streaks.

Questions in this section (E easy, M medium, H hard):

- E **First Out-of-Order Character**: fits = s[i] >= s[i-1]; return i at the first item that does not fit
- E **Group Timestamps Into Close Intervals**: close the group when the gap to the next timestamp is at least k; otherwise extend it
- M **Longest Non-Decreasing Subarray with One Replacement**: left[i] and right[i] run lengths; the best answer joins the run before and after one changed element
- M **Piano Hand Movement Grouping**: start a new group when adding the note would make max - min exceed k
- M **Find longest increasing contiguous subarray**: fits = a[i] > a[i-1]; also keep a left run and a right run per index so one change can join two runs
- M **Sum of Good Subarrays (Adjacent Diff ±1)**: keep the running length and running sum of the current up-run and down-run; add both to the total at every index
- M **Sustained-Error Time Window**: from the given second extend left and right while errors >= baseline; compare the time span with the interval
- M **Longest Non-Decreasing Subarray From Two Arrays**: two run lengths per index, one for each array you may take the value from
- M **Count Super Streaks**: sort by time; the same type within T continues the streak; when it breaks, count it if size >= N and span >= X

15. Prefix sums, tier 1: precompute once, query in O(1)

Arrays and prefix sums 8 questions: 3 easy, 5 medium, 0 hard

Spot it: "sum of a range", "balanced split", "pivot index", "equal halves", "area up to a height".

Idea: Let prefix[i] be the sum of the first i items. Then any range sum is prefix[r+1] - prefix[l], and a balance point is an index where the left total equals total minus left minus the item. The same trick works for counts of any property that adds up (equal-parity neighbours, ones in a string, area strips).

```
prefix = [0]
for x in a:
    prefix.append(prefix[-1] + x)
def range_sum(l, r): return prefix[r + 1] - prefix[l]

total = sum(a);  left = 0
for i, x in enumerate(a):                 # pivot index
    if left == total - left - x:
        return i
    left += x
```

Cost: O(n) to build, O(1) per query. **Traps:** Off by one between prefix[i] and prefix[i+1]. Use prefix[0] = 0 so a range starting at 0 works. For a removed element, look up the value that would balance both sides in a dict of first/last positions.

Follow-ups: Array changes between queries (Fenwick tree or segment tree). Circular array (double it). Remove one or k elements. Count the ways instead of finding one.

Solve these first: warm-up E, Cake Balance Line; then stretch M, Count the number of ways to split a string..

Questions in this section (E easy, M medium, H hard):

- E **Cake Balance Line**: the same balance test on the cake heights
- E **Find the Pivot Index (Balanced Split) in O(n)**: the pattern template above: left total equals total - left - x
- E **Parity Alternating Subarray Query**: pre counts neighbouring pairs of equal parity; a range alternates when pre[r] - pre[l] == 0
- M **Range Duplicate Check with Constant-Time Queries**: next[i] = next occurrence of the same value; reach[l] = smallest next over l..end; the range l..r is all distinct when ...
- M **Find a Horizontal Cut That Bisects Rectangle Area**: events by height with a running width; accumulate area strip by strip until half the total, then solve inside that strip
- M **Check pivot after removing one element**: prefix sums plus first/last position maps: look up the value whose removal would balance both sides
- M **Gold Chain Split into Two Equal Halves**: prefix sums with first/last position maps: find two cut points that leave equal halves around one removed link
- M **Count the number of ways to split a string.**: positions of the ones; the answer multiplies the gap sizes around the k-th and 2k-th one (zero ones: choose two cut ...

33. Prefix sums, tier 2: running total plus a dict of earlier totals

Arrays and prefix sums 5 questions: 0 easy, 4 medium, 1 hard

Spot it: "number of subarrays with sum k", "multiple of k", "maximum subarray", "equal endpoints".

Idea: A subarray (j, i] has sum prefix[i] - prefix[j]. So for each i you do not scan all j: you ask the dict "have I seen prefix[i] - target before, and how many times?". For a maximum subarray the dict degenerates to one number, the smallest earlier prefix (this is Kadane). For multiples of k, store the first index of each remainder prefix % k.

```
seen = {0: 1};  total = ways = 0
for x in a:                                 # count subarrays with sum k
    total += x
    ways += seen.get(total - k, 0)
    seen[total] = seen.get(total, 0) + 1

best, total, smallest = -inf, 0, 0          # maximum subarray sum
for x in a:
    total += x
    best = max(best, total - smallest)
    smallest = min(smallest, total)
```

Cost: O(n) time, O(n) space for the dict. **Traps:** Seed the dict with prefix 0 (the empty prefix). Update the dict after using it, otherwise an item pairs with itself. With a minimum length, update the "smallest earlier total" one step late.

Follow-ups: All positive numbers (then a sliding window works). Circular array. Return the subarray itself (store indices). Length limits (min or max length).

Solve these first: warm-up M, Solve array-sum and city-community problems; then stretch H, Given an integer array nums and an integer k, return true if nums has ....

Questions in this section (E easy, M medium, H hard):

- M **Solve array-sum and city-community problems**: count subarrays summing to k with the pattern template above; the second task is union-find over building cells
- M **Maximum-Sum Subarray With Equal Endpoints**: per value keep the earliest start with the smallest total before it; candidate = total now - that earlier total
- M **Maximum Subarray Sum With a Minimum Length Constraint**: the smallest earlier total is only allowed from min_len positions back, so update it one step late
- M **Find the maximum subarray sum.**: the dict is just one number: the smallest earlier total; the best subarray ends here with total - smallest (Kadane is ...
- H **Given an integer array nums and an integer k, return true if ...**: key = total % k; store the first index; a hit at least 2 positions back means a subarray whose sum is a multiple of k

20. Two pointers moving toward each other (or along two lists)

Two pointers and sliding window 7 questions: 4 easy, 3 medium, 0 hard

Spot it: "palindrome", "reverse", "sorted array and a target pair", "merge two sorted lists", "triplets that sum to zero".

Idea: Put one pointer at each end (or at the start of each list) and move whichever pointer the rule says. On a sorted array, if the pair sum is too small you must move the left pointer right, and if too large the right pointer left; this discards a whole row of pairs per step. For triplets, fix the first number and run the pair routine on the rest.

```
i, j = 0, len(a) - 1
while i < j:                                # pair with a given sum
    s = a[i] + a[j]
    if s == target: return i, j
    if s < target: i += 1
    else:          j -= 1

previous, current = None, head              # reverse a linked list
while current:
    current.next, previous, current = previous, current, current.next
```

Cost: O(n) after sorting, O(n log n) with the sort, O(1) space. **Traps:** Sort first when the problem is about values, not positions. Skip duplicates for unique triplets. Update all three pointers in one assignment when reversing links.

Follow-ups: Do it recursively. Reverse only a range or in groups of k. Allow one deletion (palindrome). Runs stored as (value, count): advance by min of the two remaining counts.

Solve these first: warm-up E, Reverse a linked list.; then stretch M, Count Unique Triplets That Sum to Zero.

Questions in this section (E easy, M medium, H hard):

E **Reverse a linked list.**: three pointers prev, current, next: point current.next back at prev, then move all three one step
E **Sort a binary array in linear time.**: left skips 0s, right skips 1s; when left sits on a 1 and right on a 0, swap them
E **Is this a valid palindrome?**: compare the two ends; skip characters that are not letters or digits; stop when the pointers cross
E **Reverse a Sentence**: swap from both ends to reverse the whole array, then reverse every word the same way
M **Find median of merged RLE arrays**: walk both sorted run lists, always taking the smaller value, adding its count until you pass the middle position
M **Run-Length Vector Storage + Dot Product**: one pointer per run list; multiply the values and take min(remaining counts) of the two current runs
M **Count Unique Triplets That Sum to Zero**: sort, fix the first number, then run the pair template on the rest; skip equal neighbours so each triple counts once

40. Sliding window: grow on the right, shrink on the left

Two pointers and sliding window 5 questions: 0 easy, 4 medium, 1 hard

Spot it: "longest/shortest subarray or substring such that ...", "at most k distinct", "minimum window", "budget of changes".

Idea: Keep a window [left, right] and a summary of what is inside (a count dict, a sum, a deque of maxima). Move right forward one item at a time and add it. While the window is too bad, remove items from the left. For a longest window, record the size after shrinking; for a shortest window, record while the window is still valid, then shrink.

```
count = Counter();  left = 0;  best = 0
for right, ch in enumerate(s):             # longest with at most k distinct
    count[ch] += 1
    while len(count) > k:
        count[s[left]] -= 1
        if count[s[left]] == 0: del count[s[left]]
        left += 1
    best = max(best, right - left + 1)

maxima = deque()                           # window maximum
for i, x in enumerate(a):
    while maxima and a[maxima[-1]] <= x: maxima.pop()
    maxima.append(i)
    if maxima[0] <= i - size: maxima.popleft()
```

Cost: O(n): each index enters and leaves the window once. **Traps:** Delete keys whose count reaches 0 or `len(count)` lies. Works only when removing items from the left can fix the window (non-negative values, monotone condition). "Exactly k" = at most k minus at most k-1.

Follow-ups: Window by elapsed time instead of count (pop while the oldest is too old). Minimum or median in the window. Count windows instead of finding the best. Many queries with different parameters.

Solve these first: warm-up M, Temperature Monitoring Data Structure; then stretch H, Necklace Cut into Two Halves with Equal D/R Counts.

Questions in this section (E easy, M medium, H hard):

M **Temperature Monitoring Data Structure**: a deque of the last n readings with a running total; a second deque of decreasing values gives the maximum
M **Footpath Beauty After Repainting**: bad = a block that is not the wanted colour; allowed = the repaint budget m; the longest window is the answer
M **Shortest Subarray with K Distinct Integers**: state = a count dict; while it holds at least k distinct values, record the window and shrink from the left
M **Find the minimum window substring.**: state = how many of each needed letter are still missing; while nothing is missing, record the window and shrink
H **Necklace Cut into Two Halves with Equal D/R Counts**: a fixed-size window of half the length slides around the doubled string, keeping a count of D

12. Greedy: scan once, carry the best so far, act only when forced

Greedy 8 questions: 3 easy, 5 medium, 0 hard

Spot it: "minimum number of ...", "maximum profit", "largest number you can form", "cover all holes", "jumps".

Idea: Make the locally best choice and never revisit it. It is correct when you can argue an exchange: any optimal answer can be changed to use your choice without getting worse. Typical shapes: carry the farthest reach and jump only when you hit the end of the current jump; scan digits from the right fixing violations; sort then take as much of the best item as fits.

```
jumps = end = farthest = 0                 # minimum jumps
for i in range(len(a) - 1):
    farthest = max(farthest, i + a[i])
    if i == end:                           # forced to jump now
        jumps += 1
        end = farthest

holes.sort();  boards = 0;  covered_to = -1   # cover holes with boards
for h in holes:
    if h > covered_to:
        boards += 1
        covered_to = h + length - 1
```

Cost: O(n) or O(n log n) when a sort is needed. **Traps:** State why greedy is valid (exchange argument) and give a counter-example where it fails (general knapsack, coins like 1, 3, 4). Ties and empty input.

Follow-ups: Many trades or a fee (turn into DP). Return the chosen indexes. Weights not all 1 (greedy may stop working: ask what changes). Huge numbers given as strings.

Solve these first: warm-up E, Alarm for the Next Upcoming Birthday; then stretch M, Jump Game II (Minimum Jumps).

Questions in this section (E easy, M medium, H hard):

- **E** **Alarm for the Next Upcoming Birthday**: minimum over people of the circular gap (their day - today) modulo 365, ties by name
- **E** **Coins on the Table**: scan the events with one counter: add on '+', subtract on '-' but never below 0
- **E** **Maximum Profit**: pick the cheapest and the dearest city with a fixed tie-break (min and max once)
- **M** **Visible People in a Line (Last Person, Then Everyone)**: scan from the end carrying the tallest person seen between; count a person when no taller one blocks the view
- **M** **Bounded Knapsack Variant**: sort items by value, highest first; take min(available count, remaining capacity) of each
- **M** **Find the Largest Monotone Increasing Number**: scan the digits right to left; when a digit is smaller than the one before it, lower the earlier digit and fill 9s ...
- **M** **Min boards to cover roof holes**: the pattern template above
- **M** **Jump Game II (Minimum Jumps)**: scan the indices carrying the farthest reach; when you hit the end of the current jump, you must jump again: jumps += ...

11. Binary search on a yes/no condition

Binary search 8 questions: 1 easy, 4 medium, 3 hard

Spot it: "minimum speed / capacity / time such that ...", "largest length such that ...", "first bad version", "median of two sorted arrays".

Idea: Find a quantity x and a test ok(x) that is monotone: false up to a threshold and true after it (or the reverse). Then binary search the threshold in O(log range) calls to ok. Write ok(x) as a simple linear scan or sum. To find the largest x that works, search the first x that fails and subtract one.

```
def first_true(low, high, ok):            # smallest x in [low, high] with ok(x)
    while low < high:
        mid = (low + high) // 2
        if ok(mid): high = mid
        else:       low = mid + 1
    return low

speed = first_true(1, max(piles),         # minimum eating speed
        lambda v: sum(-(-p // v) for p in piles) <= hours)
```

Cost: O(log(range) * cost of ok). **Traps:** Choose bounds that surely contain the answer (and say why). Use `mid = (low + high) // 2` with `high = mid` to avoid an infinite loop. Median of two arrays: binary search how many items come from the shorter array.

Follow-ups: Unknown length or huge file (double the index until the condition flips). Rate limited oracle (parallel search: more calls, fewer rounds). k sorted arrays or the k-th element (generalise the partition).

Solve these first: warm-up E, First Index Reaching a Threshold (Binary Search); then stretch H, Find the median of two sorted arrays..

Questions in this section (E easy, M medium, H hard):

- **E** **First Index Reaching a Threshold (Binary Search)**: x = an index; ok(x) = arr[x] >= k on a sorted array
- **M** **Cutting Ribbons (LC 1891)**: x = the piece length; ok(x) = the ribbons give at least k pieces; you want the LARGEST ok, so search the first x that ...
- **M** **First Bad Version with Parallel Search Follow-up**: x = the version number; ok(x) = is_bad(x)
- **M** **Solve Banana Speed and Interval Merge**: x = the eating speed; ok(x) = total hours (ceil of pile / x summed) <= h; first_true over 1..max pile
- **M** **Given a list with response time for each server, calculate ...**: x = the elapsed time; ok(x) = sum(x // time_i) >= n requests
- **H** **Median of Two Sorted Arrays (LC 4) with a Follow-up Variant**: x = how many items come from the shorter array's left side; ok(x) = the left part's largest <= the right part's smallest
- **H** **Find longest duplicated substring**: x = the substring length; ok(x) = some substring of that length appears twice (rolling hash); search the largest ok
- **H** **Find the median of two sorted arrays.**: the same partition search on the shorter array

14. bisect on a sorted list: position, range count, keep sorted

Binary search 8 questions: 1 easy, 5 medium, 2 hard

Spot it: "count in a range", "k-th largest as items stream in", "nearest value", "longest increasing subsequence".

Idea: bisect_left(a, x) gives the first index with a[i] >= x and bisect_right gives the first index with a[i] > x. Their difference counts items equal to x; two calls count a range. insort keeps a list sorted while inserting. The tails list of the longest increasing subsequence is also maintained with bisect_left: replace the first tail that is at least x, or append.

```
def count_in_range(sorted_a, low, high):   # low <= a[i] <= high
    return bisect_right(sorted_a, high) - bisect_left(sorted_a, low)

tails = []                                 # longest increasing subsequence length
for x in values:
    i = bisect_left(tails, x)
    if i == len(tails): tails.append(x)
    else:               tails[i] = x
length = len(tails)
```

```
envelopes.sort(key=lambda e: (e[0], -e[1]))  # width up, height down
```

Cost: O(log n) per search, O(n) per insort (list shift), O(n log n) for LIS. **Traps:** Strict versus non-strict increase decides bisect_left versus bisect_right. For Russian dolls, sort heights descending on equal widths. bisect on prefixes: `prefix + chr(0x10FFFF)` is a safe upper bound.

Follow-ups: Deletions too (use a Fenwick tree or a sorted container). Return the actual chain (store parent links). Count smaller numbers after self (Fenwick tree or merge sort).

Solve these first: warm-up E, Count People in a Postal Code Range; then stretch H, Count of Smaller Numbers After Self.

Questions in this section (E easy, M medium, H hard):

- E **Count People in a Postal Code Range**: sort the codes once; the answer is the pattern template above, never below 0
- M **Sliding-Window Mean After Removing the Largest K Values**: keep the window in a sorted list; insort the new value, remove the old one with bisect_left, and track the sum of the ...
- M **Streaming Insert + Find K-th Largest**: insort every new value; the k-th largest is sorted_list[-1 - k]
- M **Count Sorted Strings With a Given Prefix**: lo = bisect_left(words, prefix); hi = bisect_left(words, prefix + a very large character); the answer is hi - lo
- M **Russian Doll Envelopes**: sort by width ascending and height descending, then the longest increasing run of heights using a tails list and ...
- M **Given positions of N kids and M ice cream sellers on a 1D ...**: for each kid bisect into the sorted sellers and take the nearer of the two neighbours; the answer is the largest of ...
- H **Find Maximum Envelope Nesting**: exactly the same sort and tails list
- H **Count of Smaller Numbers After Self**: go from the right, insort each number into a sorted list; the count is bisect_left of the number before inserting (a ...

16. Sorted intervals: merge while they touch, then read gaps or search

Intervals and scheduling 8 questions: 1 easy, 5 medium, 2 hard

Spot it: "merge intervals", "do bookings conflict", "free time", "insert an interval", "union of two lists".

Idea: Sort by start. Walk once: if the next interval starts at or before the current end, extend the end with max; otherwise close the current one and start a new one. After merging, intervals are disjoint and sorted, so the gaps are the spaces between them and a point query is a bisect over the starts. A booking conflicts if the previous interval ends after the new start or the next one starts before the new end.

```
intervals.sort()
merged = [list(intervals[0])]
for start, end in intervals[1:]:
    if start <= merged[-1][1]:              # touches or overlaps
        merged[-1][1] = max(merged[-1][1], end)
    else:
        merged.append([start, end])

starts = [s for s, _ in merged]
i = bisect_right(starts, x) - 1             # the only candidate containing x
inside = i >= 0 and merged[i][1] >= x
```

Cost: O(n log n) to sort and merge, O(log n) per query. **Traps:** Decide whether touching intervals ([1,3] and [4,5], or [1,3] and [3,4]) merge. Take max of the ends, not the new end. Closed versus half-open intervals change every comparison.

Follow-ups: Insert one interval into a merged list. Remove a range. Several blocked windows (subtract each). Double booking allowed but never triple (counter dict of boundaries). Too many intervals for memory (stream and merge in chunks).

Solve these first: warm-up E, Check an Interval Against Existing Intervals for Conflicts; then stretch H, Implement a function that processes a stream of interval data..

Questions in this section (E easy, M medium, H hard):

- E **Check an Interval Against Existing Intervals for Conflicts**: bisect on the starts; a new booking conflicts if the previous one ends after its start or the next one starts before ...
- M **Find Containing Range**: merge once, keep the starts in a list, and bisect to find the one interval that could contain x
- M **Union of Sorted Interval Lists**: walk both sorted lists with two indices, always taking the smaller start next, then the same merge test
- M **Range Module (Insert and Point Query)**: keep sorted lefts and rights; adding bisects to the first interval that touches and the first that starts after, and ...
- M **Meetings Minus a Do-Not-Schedule Block**: merge first, then for each merged meeting keep the part before the blocked window and the part after it
- M **Merge Intervals**: the pattern template above
- H **Find safe travel intervals between planet influences**: turn each planet into (centre - radius, centre + radius), merge, and read the gaps (including the infinite ends)
- H **Implement a function that processes a stream of interval data.**: the Range Module add: bisect, merge the block, splice in one interval; intervals() returns the pairs

2. Sweep over events in time order

Intervals and scheduling 9 questions: 0 easy, 6 medium, 3 hard

Spot it: "minimum rooms", "maximum overlap", "skyline", "who is online at time t", "servers needed".

Idea: Turn every interval into two events (+1 at start, -1 at end), sort all events by time, and sweep while keeping a running count; the maximum count is the answer. Break ties explicitly (ends before starts if a touching meeting may reuse the room). In two dimensions, put +1/-1 at the four corners of a rectangle in a difference array and take running sums along both axes. A skyline sweep keeps a

max-heap of active heights and records a point when the top changes.

```
events = []
for start, end in meetings:
    events.append((start, 1))
    events.append((end, -1))              # -1 sorts before +1 at equal time
events.sort()
rooms = best = 0
for time, delta in events:
    rooms += delta
    best = max(best, rooms)

diff = [[0] * (w + 1) for _ in range(h + 1)]   # rectangles on a grid
for x1, y1, x2, y2 in rects:
    diff[y1][x1] += 1;  diff[y1][x2] -= 1
    diff[y2][x1] -= 1;  diff[y2][x2] += 1
```

Cost: O(n log n) for the sort, O(n) for the sweep. **Traps:** Tie-break at equal times. Meetings that cross midnight must be split in two. Merge one job's own intervals first if it should count once. Simulations (two queues at a gate) are sweeps with a rule per second.

Follow-ups: Return which room or server each interval gets (heap of free rooms). Coordinates up to 10^9 (coordinate compression or a sparse map). Online arrivals (cannot sort ahead; use a heap of end times).

Solve these first: warm-up M, Meeting Rooms / Interval Scheduling Variants; then stretch H, Compute city skyline outline.

Questions in this section (E easy, M medium, H hard):

- **M** **Meeting Rooms** / **Interval Scheduling Variants**: the pattern template above: the largest running count is the number of rooms
- **M** **Train** / **Flight Schedule Reachability**: sort legs by departure; a leg is usable if you have already reached its origin before it leaves; keep the earliest ...
- **M** **Solve Rooms and Top-K Streams**: sort starts and ends separately and walk two pointers: a start before the earliest end needs a new room; the stream ...
- **M** **Gate Entry** / **Exit Time Resolution with Tie-Break Rules**: the same two-queue simulation with the tie-break rules from the statement
- **M** **Compute servers needed for daily recurring jobs**: merge each job's own intervals first so one job counts once, then sweep all of them together
- **M** **Meeting Rooms III Variant**: split meetings that cross midnight into two pieces first, then the same sweep
- **H** **Count Overlapping Rectangle Updates on a Grid**: the same idea in two dimensions: a difference array (+1, -1, -1, +1 at the corners) and two running-sum passes
- **H** **Compute Turnstile Crossing Times**: sort people by arrival time; two queues (entering, exiting) with a rule for who goes first at each second
- **H** **Compute city skyline outline**: events at each building's left and right edge; a max-heap of active heights; record a point when the top height changes

Day 2: Strings, design and trees

21. Walk two strings at once with two indices

Strings and parsing 7 questions: 3 easy, 3 medium, 1 hard

Spot it: "is a subsequence", "lexicographic order in a custom alphabet", "can board A become board B", "similar sentences".

Idea: Keep one index per string and advance them by a rule: advance both on a match, advance only the text pointer on a mismatch (subsequence), skip blank squares in both (board transforms). A custom alphabet order is just a rank dict: compare the ranks at the first differing position. Compress words to (letter, run length) pairs when stretched letters matter.

```
def is_subsequence(needle, text):
    i = 0
    for ch in text:
        if i < len(needle) and ch == needle[i]:
            i += 1
    return i == len(needle)

rank = {c: r for r, c in enumerate(alphabet)}
def in_order(a, b):                        # alien dictionary order
    for x, y in zip(a, b):
        if x != y: return rank[x] < rank[y]
    return len(a) <= len(b)                # a prefix comes first
```

Cost: O(n + m) per pair of strings. **Traps:** The prefix rule at the end of the comparison. Skipped squares must not shift the order of the remaining letters. Pieces that move in one direction only (L left, R right) cannot cross each other.

Follow-ups: Whole list of words in an alien order (compare neighbours). Case-insensitive or accented text. Minimum number of moves. Millions of words and many queries (compress once, index by shape).

Solve these first: warm-up E, Find the First Occurrence of a Substring; then stretch H, Find Dictionary Matches for a Jammed String.

Questions in this section (E easy, M medium, H hard):

- **E** **Find the First Occurrence of a Substring**: try every start i; inside, j walks the needle; a full match returns i
- **E** **Determine if two words are sorted in lexicographic order.**: walk both words; at the first differing letter compare the letters' ranks in the given alphabet; if one word ends ...
- **E** **Return all unique substrings of a given string.**: sort all suffixes; the unique substrings are all prefixes minus the shared prefixes of neighbouring suffixes
- **M** **Can board states be transformed?**: skip the X squares in both strings; the remaining letters must match in order, and L may only move left, R only right
- **M** **Decide string transform with directional tokens and walls**: the same walk with dots as blanks and the movement rule from the statement (walls block)
- **M** **Determine if two sentences are similar.**: zip the two sentences; each pair of words must be equal or listed as similar (store pairs in both directions)
- **H** **Find Dictionary Matches for a Jammed String**: compress every word to (letter, run length); a query matches when the letters agree and each run is equal or stretched ...

43. Split into pieces, fix each piece, join (and word wrap)

Strings and parsing 5 questions: 1 easy, 2 medium, 2 hard

Spot it: "snake_case to camelCase", "format a number", "justify text", "wrap at width".

Idea: Break the text at its separators, transform each piece with a tiny function, and join. For wrapping, add words greedily while current length + 1 + word length fits in the width, otherwise start a new line. For full justification, spread the spaces of a line over its gaps (the leftmost gaps get the remainder) and left-align the last line.

```
def to_camel(name):
    first, *rest = name.split('_')
    return first + ''.join(p.capitalize() for p in rest)

lines, current = [], []
for word in words:
    if current and len(' '.join(current + [word])) > width:
        lines.append(current);  current = []
    current.append(word)
lines.append(current)
```

Cost: O(total characters). **Traps:** Leading, trailing and repeated separators. A word longer than the width. Do not use += on strings in a long loop; collect pieces and join once.

Follow-ups: Convert back (camel to snake). Locale grouping (12,34,567). Words with pixel widths. Minimise raggedness instead of greedy packing (that is a DP over line breaks).

Solve these first: warm-up E, Implement string formatting and request tracking tasks; then stretch H, Implement a function to justify a list of words within a given width..

Questions in this section (E easy, M medium, H hard):

- **E** **Implement string formatting and request tracking tasks**: split, fix each piece, join; the request-tracking part is a small dict

M **Snake Case to Camel Case with Repository-Wide Refactor**: the same function applied to every identifier found in the files
M **Google Doc Line Wrap (Word Wrap with Whitespace and Newlines)**: count lines: go paragraph by paragraph, add a word if it fits (current + 1 + length <= width), else start a new line
H **Convert Valid Snake Case to Lower Camel Case**: reformat(): split on '_', keep the first piece, capitalise the others
H **Implement a function to justify a list of words within a ...**: pack_lines for the words, then spread the extra spaces over the gaps (the left gaps get the remainder); the last line ...

24. Digits and characters: peel one at a time and carry

Math and bit tricks 6 questions: 1 easy, 3 medium, 2 hard

Spot it: "add two big numbers", "palindrome number", "monotone digits", "caesar cipher", "k-th permutation".

Idea: Work with one digit at a time using n % 10 (last digit) and n // 10 (drop it), or with ord/chr arithmetic for letters. Addition keeps a carry: total = a + b + carry, digit = total % 10, carry = total // 10. For the k-th permutation, divide k-1 by (n-1)!, (n-2)!, ... to choose each digit from the list of unused digits (factorial number system).

```
def add_digits(a, b):                       # lists, most significant first
    i, j, carry, out = len(a) - 1, len(b) - 1, 0, []
    while i >= 0 or j >= 0 or carry:
        total = carry + (a[i] if i >= 0 else 0) + (b[j] if j >= 0 else 0)
        out.append(total % 10);  carry = total // 10
        i -= 1;  j -= 1
    return out[::-1]

def shift(ch, k):                           # caesar
    return chr((ord(ch) - 97 + k) % 26 + 97)
```

Cost: O(number of digits). **Traps:** A final carry creates a new leading digit. Negative k works with % in Python. A number palindrome only needs the second half reversed.

Follow-ups: Numbers as linked lists (least significant first makes addition easy). Another base. Break a cipher given only ciphertext (frequency analysis). Position of a permutation (inverse of the factorial method).

Solve these first: warm-up E, Caesar Cipher: Recover Key, Encrypt, and Decrypt; then stretch H, Permutation Sequence.

Questions in this section (E easy, M medium, H hard):

E **Caesar Cipher: Recover Key, Encrypt, and Decrypt**: per letter: (ord(ch) - 97 + k) % 26 + 97; the key comes from one letter pair; decrypt uses -k
M **Solve several coding interview problems**: the big-number addition on strings, with int(ch) per digit; the other parts are list-meeting and overlap sweeps
M **Check Whether Digits Are Monotone Increasing**: peel digits from the right; each digit must not exceed the previous one you saw
M **Add two big integers from digit lists**: the pattern template above
H **Check Digits and Combine Ranges**: palindrome number: reverse only the second half with n % 10 and n // 10 and compare with the first half
H **Permutation Sequence**: factorial number system: divide k - 1 by (n - 1)!, (n - 2)!, ... to pick each digit from the list of unused digits

42. Build or transform a grid with index arithmetic

Grids and simulation 5 questions: 1 easy, 4 medium, 0 hard

Spot it: "rotate a matrix", "range sum in a 2D matrix", "draw a diamond", "fill in snake order", "block minimum".

Idea: Derive the formula for where each cell goes and write it directly. Rotating clockwise: new[r][c] = old[rows - 1 - c][r], or in Python zip(*matrix[::-1]). A 2D prefix table p[r+1][c+1] = cell + p[r][c+1] + p[r+1][c] - p[r][c] answers any rectangle with four lookups by inclusion and exclusion. Block grouping uses (r // k, c // k) as the key.

```
prefix = [[0] * (cols + 1) for _ in range(rows + 1)]
for r in range(rows):
    for c in range(cols):
        prefix[r + 1][c + 1] = (matrix[r][c] + prefix[r][c + 1]
                                + prefix[r + 1][c] - prefix[r][c])
def rect_sum(r1, c1, r2, c2):               # inclusive corners
    return (prefix[r2 + 1][c2 + 1] - prefix[r1][c2 + 1]
            - prefix[r2 + 1][c1] + prefix[r1][c1])

rotated = [list(row) for row in zip(*matrix[::-1])]
```

Cost: O(rows * cols) to build, O(1) per query. **Traps:** Do not rotate non-square matrices in place. Check the formula on a 2 by 3 example by hand. Blocks that do not tile the grid exactly need a decision (pad or drop).

Follow-ups: Rotate by k quarter turns (k % 4). Cell updates between queries (2D Fenwick tree). Detect impossible recipes by cycle (topological sort).

Solve these first: warm-up E, Filled ASCII Diamond; then stretch M, Rotate Any m×n 2D Matrix.

Questions in this section (E easy, M medium, H hard):

E **Filled ASCII Diamond**: row i has pad = |i - R| underscores on each side and 2 * (R - pad) + 1 x's in the middle
M **Connected Crop Arrangement on a Grid**: fill the cells in snake order (left to right, then right to left) so equal crops stay in one connected strip
M **Solve matrix groups and recipe inventory**: block (r // k, c // k) holds the minimum of every cell that maps to it; the recipe part is a topological sort
M **Range Sum Query 2D (Immutable Matrix)**: prefix table p[r + 1][c + 1] = cell + p[r][c + 1] + p[r + 1][c] - p[r][c]; a query is four lookups
M **Rotate Any m×n 2D Matrix**: clockwise: new[r][c] = old[rows - 1 - c][r] on a cols-by-rows result (or zip(*matrix[::-1]))

22. Simulate a machine: a class with state and one branch per event

Grids and simulation 7 questions: 1 easy, 5 medium, 1 hard

Spot it: "design tic-tac-toe", "elevator system", "iterator over lists", "game of life", "scorekeeper".

Idea: Name the state (board, floor, queues, counters), write one method per kind of input, and make every method update the state in one place. For tic-tac-toe, keep counters per player for rows, columns and both diagonals so a win check is O(1). For game of life in place, encode old and new state in two bits of each cell so every cell is computed from the old state, then shift.

```
class TicTacToe:
    def __init__(self, n):
        self.n = n
        self.rows = defaultdict(int);  self.cols = defaultdict(int)
        self.diag = defaultdict(int);  self.anti = defaultdict(int)
    def move(self, r, c, player):
        for table, key in ((self.rows, (player, r)), (self.cols, (player, c))):
            table[key] += 1
            if table[key] == self.n: return player
        if r == c:               self.diag[player] += 1
        if r + c == self.n - 1:  self.anti[player] += 1
        if self.n in (self.diag[player], self.anti[player]): return player
        return 0
```

Cost: O(1) per move for tic-tac-toe, O(cells) per step for life. **Traps:** Reject moves on a taken cell if the spec says so. Neighbour counts in life must read the old bit (value & 1). Elevator: define the cost of assigning a request to a car.

Follow-ups: Undo the last move (stack of moves). Infinite or sparse board (set of live cells). Wrap-around edges. Fairness, starvation and failures for schedulers.

Solve these first: warm-up E, Mario-Style Score Tracker (OOP); then stretch H, Simulate In-Place Cellular State Updates.

Questions in this section (E easy, M medium, H hard):

- **E** **Mario-Style Score Tracker (OOP)**: the pattern template above: an event name decides how score, lives and size change
- **M** **Tic-Tac-Toe System with K Players and 3-in-a-Row Win**: store the board; after each move count the run through the new cell in four directions and compare with the target ...
- **M** **Design Tic-Tac-Toe With K Players**: per player Counters for rows, columns and both diagonals; a move wins when a counter reaches n
- **M** **Implement iterator merging lists with filtering**: keep a position per list and a turn counter; advance() looks for the next unblocked, unseen value and stores it for ...
- **M** **Infinite Deck Card Duel**: an infinite deck is just a random draw; play rounds, compare the highest cards, and decide the champion by the lead in ...
- **M** **Design an elevator control system**: each car keeps floor, direction and stops; a request goes to the car with the lowest cost; step() moves every car one ...
- **H** **Simulate In-Place Cellular State Updates**: encode old and new state in two bits of each cell so every cell is computed from the old state; shift at the end

18. A dict for lookup plus an ordered container for who is next

Data structure and object design 8 questions: 1 easy, 6 medium, 1 hard

Spot it: "LRU cache", "waitlist", "round robin scheduler", "remove the cheapest", "undo / redo".

Idea: A dict gives O(1) access by id; a second structure gives order: a deque for first-in-first-out, a doubly linked list or OrderedDict for recency, a heap for cheapest. Deleting from the ordered side is expensive, so use lazy deletion: only remove the id from the dict, and when you pop from the order structure, skip entries that are no longer alive. Undo and redo are two stacks; a batch is one stack entry.

```
class LRU:
    def __init__(self, capacity):
        self.capacity, self.data = capacity, OrderedDict()
    def get(self, key):
        if key not in self.data: return -1
        self.data.move_to_end(key)         # most recent at the end
        return self.data[key]
    def put(self, key, value):
        self.data[key] = value
        self.data.move_to_end(key)
        if len(self.data) > self.capacity:
            self.data.popitem(last=False)  # evict least recent

waiting = deque();  alive = {}             # lazy deletion waitlist
def next_party():
    while waiting and waiting[0] not in alive: waiting.popleft()
    return waiting.popleft() if waiting else None
```

Cost: O(1) per operation for LRU; O(log n) with a heap. **Traps:** Every access must refresh recency. Lazy deletion needs a unique ticket per insert (a name can rejoin). Say what the order is (arrival, price, deadline).

Follow-ups: Thread safety (a lock). TTL expiry. LFU instead of LRU (frequency buckets). Best fit instead of first fit (segment tree over sizes). Limit the undo history.

Solve these first: warm-up E, Implement Batched Undo/Redo Layer; then stretch H, Implement LRU Cache..

Questions in this section (E easy, M medium, H hard):

E **Implement Batched Undo/Redo Layer**: two stacks (undo, redo) of batches; a batch records (old value, new value) per property; undo applies the old values ...
M **Design a restaurant waitlist system**: a segment tree of party sizes gives the first entry that fits in O(log n); the dict maps ids to slots
M **Implement Python's itertools.tee**: one deque per consumer; whoever runs out pulls the next value from the source and appends it to every queue
M **Restaurant Waitlist: join / delete / find_first_match**: a deque per party size with tickets; delete only removes the name from the dict; find_first_match skips dead tickets
M **Store With Cheapest-Item Removal**: the order is a heap keyed by (price, timestamp); remove_cheapest pops it
M **Round-Robin Job Scheduler**: a deque of [job, time left]; pop, run for min(quantum, left); push back at the end if unfinished, else record the ...
M **Aggregate FizzBuzz Offer Matches**: a deque of offers per type; a new offer matches against the front of the opposite queue, partial quantities stay at the ...
H **Implement LRU Cache.**: dict key -> node plus a doubly linked list; every get/put moves the node to the front; evict from the back (or use ...

41. Parent pointers: store who is above each node and climb

Data structure and object design 5 questions: 1 easy, 4 medium, 0 hard

Spot it: "lowest common ancestor with parent links", "validate a tree", "employee hierarchy", "is this a root".

Idea: Store parent[node] (and sometimes a set of children). Climbing is a loop following parents. Lowest common ancestor with parent links: advance two pointers, and when one falls off the top restart it at the other node; they meet at the ancestor after at most len(a) + len(b) steps. Validating a parent array: climb from every node with three states (unseen, on this walk, verified); meeting your own walk is a cycle, and there must be exactly one root.

```
def lca(a, b):
    p, q = a, b
    while p is not q:
        p = p.parent if p else b
        q = q.parent if q else a
    return p

state = {}                                  # 0 unseen, 1 on this walk, 2 verified
def has_cycle(start):
    walk = []
    node = start
    while node is not None and state.get(node, 0) == 0:
        state[node] = 1;  walk.append(node);  node = parent[node]
    bad = node is not None and state.get(node) == 1
    for n in walk: state[n] = 2
    return bad
```

Cost: O(depth) per climb, O(n) for validating all nodes. **Traps:** The root has no parent (None). Check for exactly one root and for nodes that are not reachable. Deleting a manager means re-attaching the reports.

Follow-ups: Many LCA queries on a static tree (binary lifting or Euler tour). Move an employee and reject cycles. Nodes might not share a tree (return None).

Solve these first: warm-up E, Tree Node.; then stretch M, Lowest Common Ancestor of a Binary Tree III.

Questions in this section (E easy, M medium, H hard):

E **Tree Node.**: the SQL version of the same idea: p_id is the parent pointer, NULL means root, being someone's parent means inner, else ...
M **Employee Hierarchy Add / Delete**: manager[emp] and a set of reports per manager; delete re-attaches the reports to the deleted person's manager
M **Validate parent array forms a tree**: climb from every node with three states (unseen, on this walk, verified); meeting your own walk is a cycle; there must ...
M **Root a Tree So It Becomes a Binary Tree**: the root has no parent so it needs degree <= 2; everyone else spends one edge on its parent; answer = the first node of ...
M **Lowest Common Ancestor of a Binary Tree III**: two pointers climb; when one falls off the top it restarts at the other node, so both meet at the common ancestor

37. Random pick without repeats: swap with the last and pop

Data structure and object design 5 questions: 0 easy, 4 medium, 1 hard

Spot it: "random set with O(1) delete", "shuffle", "unique random generator", "no repeat within k songs".

Idea: To delete from the middle of a list in O(1), swap the victim with the last element, update the position dict, and pop. Random pick is then a random index into the list. A shuffle is Fisher-Yates: for i from the end, swap a[i] with a random a[j] where j <= i. A huge pool 0..n-1 can be lazy: keep a dict of only the swapped slots.

```
class RandomSet:
    def __init__(self):  self.items, self.where = [], {}
    def add(self, x):
        if x in self.where: return False
        self.where[x] = len(self.items);  self.items.append(x);  return True
    def remove(self, x):
        if x not in self.where: return False
        i, last = self.where[x], self.items[-1]
        self.items[i] = last;  self.where[last] = i
        self.items.pop();  del self.where[x];  return True
    def pick(self):  return random.choice(self.items)

for i in range(len(a) - 1, 0, -1):          # Fisher-Yates
    j = random.randint(0, i);  a[i], a[j] = a[j], a[i]
```

Cost: O(1) per add, remove and pick. **Traps:** Update the position of the moved element before deleting. A shuffle that picks j from the whole array is biased. Raise or return a sentinel when the pool is empty.

Follow-ups: Duplicates allowed (map value to a set of positions). Reset in O(1) (keep a counter, do not rebuild). Only a coin flip API exists (rejection sampling). Thread safety.

Solve these first: warm-up M, Design set with O(1) random access; then stretch H, Shuffle array using random integer API.

Questions in this section (E easy, M medium, H hard):

- M **Design set with O(1) random access**: add appends and records the position; delete swaps with the last item and pops; get_random picks a random index
- M **Random Generator Without Repeats**: the pattern template above: values 0..n-1, every call returns a new one; raise when empty
- M **Random Song Shuffler With a No-Repeat Window**: the same pool, plus a deque of the last k songs; the played song leaves the pool and returns to it when it drops off ...
- M **Unique Random Number Generator**: the lazy version for huge n: a dict 'moved' stores only the swapped slots, so memory stays O(calls)
- H **Shuffle array using random integer API**: Fisher-Yates: for i from the end, j = rand_int(0, i), swap a[i] and a[j]

27. Nested structures: a stack of contexts, or recursion with a cache

Stacks and queues 5 questions: 0 easy, 5 medium, 0 hard

Spot it: "decode string 3[a2[c]]", "evaluate expression", "nested list weight", "expand placeholders".

Idea: Each open bracket starts a new context, each close bracket finishes it and combines the result into the one below. Keep a stack of (partial result, pending multiplier) and push on open, pop on close. For recursive definitions (templates referring to keys), resolve with a cache and a "currently resolving" set so a cycle is detected instead of looping forever.

```
stack, current, number = [], '', 0
for ch in s:
    if ch.isdigit():     number = number * 10 + int(ch)
    elif ch == '[':      stack.append((current, number));  current, number = '', 0
    elif ch == ']':
        previous, times = stack.pop();  current = previous + current * times
    else:                current += ch

def resolve(key, cache, visiting):
    if key in cache: return cache[key]
    if key in visiting: raise ValueError('cycle')
    visiting.add(key)
    cache[key] = expand(values[key], cache, visiting);  visiting.discard(key)
    return cache[key]
```

Cost: O(output size) for decoding, O(n) for parsing. **Traps:** Multi-digit numbers. The decoded string may be enormous (compute its length or the k-th character without building it). Unknown keys and unmatched brackets need a defined behaviour.

Follow-ups: Recursion depth (iterate with an explicit stack). Division and precedence in expressions. Variables and variadic functions. Inverse weighting (deeper counts less).

Solve these first: warm-up M, Decode String; then stretch M, Nested Function Expression Evaluator.

Questions in this section (E easy, M medium, H hard):

- M **Decode String**: the pattern template above
- M **Template Variable Expansion with Follow-Ups**: resolve(name) with a cache and a 'being resolved' set; meeting a name that is being resolved is a cycle
- M **Expand Recursive Key-Value Placeholders in a Template**: regex-find {key}, expand each key recursively with a cache and a visiting set to stop cycles
- M **Nested List Weight Sum (LC 339) with Follow-Ups**: depth counter: '[' adds 1, ']' subtracts 1; each number adds value * depth (read digits and a minus sign as you go)
- M **Nested Function Expression Evaluator**: tokenise, then a recursive parse(i): a number returns itself; a name reads its arguments until ')' and applies the ...

25. Look at the input once and keep only a small summary

Streaming, scale and concurrency 6 questions: 0 easy, 4 medium, 2 hard

Spot it: "file does not fit in memory", "stream", "minimise the number of oracle calls", "two logs".

Idea: Decide what the smallest summary is that still answers the question: a running maximum, a set of ids from the smaller file, a size-k heap, a count per key. Stream the large input past it. When a test oracle is expensive, use group testing: test a whole group in one call and split only the groups that fail, or binary search a prefix to find the first failure.

```
best = None                                 # maximum in a huge file
with open(path) as handle:
    for line in handle:
        value = int(line)
        if best is None or value > best: best = value

small_ids = {line.strip() for line in open(small_file)}
shared = [line for line in open(big_file) if line.strip() in small_ids]

def find_bad(group):                        # group testing
    if not run_test(group): return []
    if len(group) == 1: return group
    mid = len(group) // 2
```

```
    return find_bad(group[:mid]) + find_bad(group[mid:])
```

Cost: O(n) time, memory proportional to the summary, not the input. **Traps:** Never call `.read()` or `.readlines()` on a huge file. Say how you would handle both files being too big (partition by hash of the key, process partition by partition). Edge cases: empty file, blank lines, ties.

Follow-ups: Scale to many machines (map and reduce by key). Failures and retries. Top K instead of max (heap of size K). Is it I/O bound or CPU bound?

Solve these first: warm-up M, Streaming Points: Emit K Within Distance; then stretch H, Find maximum value in huge text file.

Questions in this section (E easy, M medium, H hard):

- M **Streaming Points: Emit K Within Distance**: keep the points in a sorted list; each new point is inserted with bisect and only windows around it are re-checked
- M **Two-Day Log Intersection under Memory Limit**: dict object -> set of event codes for each day; keep the objects that appear on both days with at least two distinct ...
- M **Find shared objects across two log files**: load only the smaller file's object ids into a set, then stream the big file and keep ids found in the set
- M **Minimize calls to find all bad test pairs**: the same group testing: put a whole group in one call and split only the groups that fail
- H **Find All Bad Pairs Using runTest Oracle**: for each test, binary search the prefix of earlier tests where runTest first fails; that test is a bad partner; repeat ...
- H **Find maximum value in huge text file**: the pattern template above; the file is never loaded as a whole

3. Tree recursion, tier 1: visit a node, ask children, report upward

Binary trees and BSTs 9 questions: 3 easy, 4 medium, 2 hard

Spot it: "height", "diameter", "right side view", "level order", "delete leaves", "evaluate expression tree".

Idea: Write a function that takes a node and returns one number or object about the subtree (height, value, whether it is valid). Call it on the children first (post-order), combine, and update a global answer if the answer is not the return value. Diameter: return 1 + max(left, right) but update best with left + right. For per-level problems pass depth down and store the first or last node seen at each depth.

```
best = 0
def height(node):
    global best
    if node is None: return 0
    left, right = height(node.left), height(node.right)
    best = max(best, left + right)         # diameter in edges
    return 1 + max(left, right)

def right_view(node, depth, view):
    if node is None: return
    if depth == len(view): view.append(node.val)
    right_view(node.right, depth + 1, view)    # right first
    right_view(node.left, depth + 1, view)
```

Cost: O(n) time, O(height) stack space. **Traps:** Say whether lengths count nodes or edges. A chain of 100,000 nodes overflows Python recursion: convert to an explicit stack. Return and update are two different numbers.

Follow-ups: N-ary tree (loop over children). Is the tree balanced (return height or -1). Return the path itself (store the best endpoints). Delete subtrees that match a predicate.

Solve these first: warm-up E, Find right-side view of binary tree; then stretch H, Delete an Entire Binary Tree.

Questions in this section (E easy, M medium, H hard):

- E **Find right-side view of binary tree**: pass depth down; the first node you reach at a new depth (visiting right before left) is the one you can see
- E **Given two trees, determine if the pre-order traversals of two ...**: walk both trees in preorder with an explicit stack (push right, then left) and compare value by value; stop at the ...
- E **Calculate the height of a binary tree.**: no update needed; return 1 + max(left, right) and read the value at the root
- M **Expression Tree Single-Leaf Mutation**: post-order evaluation (children first, then apply the operator); to test one leaf flip, re-evaluate with that leaf ...
- M **Longest Consecutive Path in Binary Tree**: pass the parent's value and the run length down; extend the run when value == parent + 1, else restart at 1
- M **Implement zigzag level-order traversal**: collect values by depth; reverse every odd level (or fill each level from the other end)
- M **Given the root of a binary tree, return the length of the ...**: best = max(best, left + right); return 1 + max(left, right) (the diameter)
- H **Repeatedly Delete Leaves Matching a Target Value From a ...**: set node.left = dfs(node.left) and node.right likewise; return None when the node became a leaf with the target value
- H **Delete an Entire Binary Tree**: post-order: free the children first, then the node itself

36. Tree recursion, tier 2: pass a rule down, return a node or yes/no up

Binary trees and BSTs 5 questions: 1 easy, 4 medium, 0 hard

Spot it: "valid BST", "lowest common ancestor", "in-order successor", "mode in a tree".

Idea: Pass information down as arguments (the allowed value window low and high for BST validation) and return a verdict or a node up. A BST in-order walk is sorted, so equal values are neighbours (running count gives the mode) and the successor of x is found by descending: go left after a hit, right otherwise. Lowest common ancestor: return the node if it is p or q; a node that receives an answer from both sides is the ancestor.

```
def is_bst(node, low=float('-inf'), high=float('inf')):
```

```
    if node is None: return True
    if not low < node.val < high: return False
    return is_bst(node.left, low, node.val) and is_bst(node.right, node.val, high)

def lca(node, p, q):
    if node is None or node is p or node is q: return node
    left, right = lca(node.left, p, q), lca(node.right, p, q)
    return node if left and right else left or right
```

Cost: O(n) for the walks, O(height) for a BST descent. **Traps:** Compare against the whole window, not only the parent. Decide how duplicates are placed. p or q may be missing from the tree (verify with a count).

Follow-ups: Not a BST (generic traversal). O(1) space (Morris traversal). Recover a BST with two swapped nodes. Deep trees (iterative).

Solve these first: warm-up E, Next Birthday (BST In-Order Successor with Wraparound); then stretch M, Determine if a given binary tree is a binary search tree (BST)..

Questions in this section (E easy, M medium, H hard):

- E **Next Birthday (BST In-Order Successor with Wraparound)**: descend(): go left after a hit (a closer candidate may exist), right otherwise; if no candidate exists wrap around to ...
- M **Find mode in a trinary search tree**: in-order walk (left, middle, right); equal values are neighbours, so one running count finds the mode(s)
- M **Next Upcoming Birthday in a Family Tree**: walk the ancestors with a stack and a seen set, keep the best birthday by the circular distance from today
- M **Lowest Common Ancestor**: dfs returns the node if it is p or q; a node that gets an answer from both sides is the ancestor
- M **Determine if a given binary tree is a binary search tree ...**: check(): pass lo and hi bounds down; fail when a value leaves its allowed window

31. Tree DP: build a parent order once, add up from the leaves

Binary trees and BSTs 5 questions: 0 easy, 3 medium, 2 hard

Spot it: "total size of a folder", "sum of distances from every node", "time to inform everyone", "cheapest cut".

Idea: Get an order where parents come before children (BFS or DFS from the root). Sweep it backwards to compute subtree values (size, sum, best) from children, then sweep it forwards to push information down. Distance sums for every root (rerooting): down[v] is the sum inside the subtree of v, and ans[child] = ans[parent] - size[child] + (n - size[child]).

```
order, parent = [root], {root: None}
for node in order:                          # BFS order, parents first
    for child in children[node]:
        parent[child] = node;  order.append(child)

size = {v: 1 for v in order};  down = {v: 0 for v in order}
for v in reversed(order):                   # leaves to root
    if parent[v] is not None:
        size[parent[v]] += size[v]
        down[parent[v]] += down[v] + size[v]

answer = {root: down[root]}
for v in order[1:]:                         # root to leaves (rerooting)
    answer[v] = answer[parent[v]] - size[v] + (n - size[v])
```

Cost: O(n), and no recursion so a long chain is safe. **Traps:** Prove the reroot formula with a two node tree. Hard links count a shared item once (seen set). Serve the child with the longest time first when informing in rounds.

Follow-ups: Weighted edges. Maximum distance instead of sum. Which edges to cut. The tree is queried often while files change (store subtree sizes and update upward).

Solve these first: warm-up M, Compute File System Total Size; then stretch H, Tree Distance Sum (Re-root DP).

Questions in this section (E easy, M medium, H hard):

- M **Compute File System Total Size**: add each file's size while walking down from the root with a stack; a shared directory is counted once (seen set)
- M **Minimum-Cost Tree Disconnection**: backwards sweep with two numbers per node: the best cost when the node is cut off, and the cost if it is free
- M **Given a large n-ary tree where the root node can pass ...**: t[v] = max over children (rank among children sorted by time, longest first, + t[child]); read t at the root
- H **Compute distance-sum from every tree node**: down[v] = sum of distances to v's subtree; then a forward sweep: ans[child] = ans[parent] - size[child] + (n - ...
- H **Tree Distance Sum (Re-root DP)**: exactly the same two sweeps (this is the re-rooting version of the question)

38. Build and serialize: write out a structure, read it back with the same walk

Binary trees and BSTs 5 questions: 0 easy, 3 medium, 2 hard

Spot it: "serialize / deserialize", "construct tree from preorder and inorder", "print a directory tree", "encode a list of strings".

Idea: Pick a format that makes decoding unambiguous and decode with the same traversal order you used to encode. Strings: write length, a separator, then the text, so any character may appear inside. Preorder plus inorder: the first preorder item is the root and its position in the inorder list splits left from right. N-ary tree: write the value and the child count. Graph: write one line per node with the ids of neighbours, create all nodes first, then link.

```
def encode(strings):
    return ''.join(f'{len(s)}#{s}' for s in strings)
```

```
def decode(text):
    out, i = [], 0
    while i < len(text):
        j = text.index('#', i);  length = int(text[i:j])
        out.append(text[j + 1:j + 1 + length]);  i = j + 1 + length
    return out

def build(preorder, inorder):
    index = {v: i for i, v in enumerate(inorder)}
    it = iter(preorder)
    def make(low, high):
        if low > high: return None
        node = Node(next(it));  mid = index[node.val]
        node.left, node.right = make(low, mid - 1), make(mid + 1, high)
        return node
    return make(0, len(inorder) - 1)
```

Cost: O(n) time and space. **Traps:** Build left before right when consuming a preorder iterator. Values may repeat in preorder plus inorder construction (ambiguous). Graph: handle cycles with a seen map.

Follow-ups: Postorder plus inorder. Level order encoding. Binary encoding for compactness. Why not JSON (length prefix works for any bytes).

Solve these first: warm-up M, Print a Directory Tree; then stretch H, Implement a decode and encode function of a general N-ary tree..

Questions in this section (E easy, M medium, H hard):

- M **Print a Directory Tree**: build nested dicts from the path parts, then print them with a recursive walk that indents by depth
- M **Construct a binary tree given its preorder and inorder ...**: the first preorder value is the root; its position in a value-to-index map of the inorder list splits left from right
- M **Write functions to serialize and deserialize a list of ...**: write len + '#' + text for each string; read the length, then take exactly that many characters
- H **Serialize and Deserialize a Graph**: BFS with a seen set; write one line per node: value, then the values of its neighbours; rebuild by creating all nodes ...
- H **Implement a decode and encode function of a general N-ary ...**: write value and child count for each node (preorder); read the count to know how many children to build

17. Trie: nested dicts, one character per level

Tries 8 questions: 0 easy, 7 medium, 1 hard

Spot it: "autocomplete", "words with a prefix", "longest match tokenizer", "URL router", "dictionary substrings".

Idea: Each node is a dict from character to child; a marker (END) says a word stops here. Insert walks and creates; search walks and checks END; prefix queries walk to the prefix node and collect everything below. For longest-match replacement, walk as far as the trie goes from each text position, remember the last END seen, and jump past it. A router uses one level per path segment with a literal child and one wildcard child.

```
END = '$'
def insert(root, word):
    node = root
    for ch in word:
        node = node.setdefault(ch, {})
    node[END] = True

def longest_match(root, text, start):
    node, last = root, -1
    for i in range(start, len(text)):
        if text[i] not in node: break
        node = node[text[i]]
        if END in node: last = i
    return last                              # -1 when nothing matches
```

Cost: O(length of word) per operation, O(total characters) space. **Traps:** END must be distinguishable from a real character. Deleting needs pruning of empty nodes. For prefix lookups on a static list, sorting plus bisect is lighter than a trie.

Follow-ups: Top 3 completions (store them per node). Wildcard "." search (recurse over all children). Fuzzy match with one error. Huge dictionaries (sharding, compressed trie).

Solve these first: warm-up M, URL Request Handler Router; then stretch H, Implement Longest-Match Text Replacement.

Questions in this section (E easy, M medium, H hard):

- M **URL Request Handler Router**: one trie level per URL segment; a node has literal children and one wildcard child; try the literal first
- M **Trie Prefix Search** / **Autocomplete**: walk to the prefix node; return the words under it (autocomplete)
- M **Build Prefix Lookup with a Trie**: build_trie, then walk to the prefix node and collect every word below it
- M **Greedy Longest-Match Dictionary Tokenizer**: the same longest-match walk, with a priority rule to choose between equal-length matches
- M **Check if all substrings are dictionary words**: for every start walk the trie; a substring of length 3 or more that leaves the trie or lacks END fails the check
- M **Return dictionary words matching a prefix**: no trie needed: sort the words and bisect to the first match, then read forward while startswith(prefix)
- M **Implement Trie Insert and Exact-Word Search**: insert is build_trie; search is walk and then END in node
- H **Implement Longest-Match Text Replacement**: at each text position walk the trie as far as it goes and remember the last END; replace the longest match and jump ...

Day 3: Recursion, dynamic programming, heaps and graphs

7. Backtracking: choose, recurse, undo

Recursion and backtracking 9 questions: 0 easy, 8 medium, 1 hard

Spot it: "all permutations / combinations", "generate parentheses", "count paths of length k", "can this hand win", "assign people to tasks".

Idea: Build the answer one choice at a time. At each step loop over the legal choices, apply one, recurse, then undo it so the next choice starts from the same state. Prune as early as possible: only add a close bracket when closed < opened, always try the smallest remaining tile first for Mahjong. If you only need the count, return numbers instead of storing every solution. Matching problems use an augmenting path: if the preferred slot is taken, recursively ask its owner to move.

```
def parentheses(n):
    out = []
    def build(current, opened, closed):
        if len(current) == 2 * n:
            out.append(current);  return
        if opened < n:  build(current + '(', opened + 1, closed)
        if closed < opened:  build(current + ')', opened, closed + 1)
    build('', 0, 0)
    return out

def permutations(items):
    out, chosen, used = [], [], [False] * len(items)
    def build():
        if len(chosen) == len(items): out.append(chosen[:]);  return
        for i, x in enumerate(items):
            if used[i]: continue
            used[i] = True;  chosen.append(x)
            build()
            chosen.pop();  used[i] = False        # undo
    build()
    return out
```

Cost: Exponential: n! permutations, 2^n subsets, Catalan(n) bracket strings. Pruning cuts the constant. **Traps:** Copy the list when you store it (`chosen[:]`). Undo exactly what you did. Skip equal neighbours (after sorting) to avoid duplicate results. Use a bitmask for visited sets when n is small.

Follow-ups: Count only. Iterative version with an explicit stack. Next lexicographic permutation. k-th permutation directly (factorial number system). Priorities or capacities in matching.

Solve these first: warm-up M, Mahjong Winning Hand Detection (14 Tiles); then stretch H, Matrix Flower Placement with House Adjacency.

Questions in this section (E easy, M medium, H hard):

- M **Mahjong Winning Hand Detection (14 Tiles)**: take the smallest tile; try it as a triple, then as the start of a run; undo the counts after each try
- M **Count Prefix-Matching Simple Paths in a Labeled Graph**: extend the path along edges whose label matches the next query letter, with a bitmask of visited nodes; count per depth
- M **Determine if a 14-tile hand is winning**: the same try-triple-then-run recursion over a count array, after choosing the pair
- M **American Football Score Combinations**: choices are the scoring plays from index i onward (the same play may repeat); complete when the remaining score is 0
- M **Count Passcode Patterns on a Keypad**: the choice is the next adjacent unused key; complete at k keys; count instead of storing
- M **Match people to questions using tags and priorities**: try each question of a person; if it is taken, recursively ask its owner to move (augmenting path); undo by not ...
- M **Generate Parentheses**: choices are '(' and ')'; allowed when opened < n, and ')' only when closed < opened; complete at length 2n
- M **Permutations**: the pattern template above
- H **Matrix Flower Placement with House Adjacency**: place row by row; reject a row early when the houses two rows above are still not next to a flower

39. Games: win or lose with a cache, and guessing with feedback

Recursion and backtracking 5 questions: 0 easy, 1 medium, 4 hard

Spot it: "who wins with optimal play", "merge stacks game", "guess the secret word", "Wordle strategy".

Idea: Describe the whole game position as a hashable state (a sorted tuple of stacks). A position is winning if some move leads to a losing position for the opponent; cache the answer per state so each state is solved once. For guessing games, choose the guess that splits the remaining candidates into the most even feedback groups (minimise the size of the largest group), then keep only candidates consistent with the feedback received.

```
@lru_cache(maxsize=None)
def wins(state):                          # state: sorted tuple of stack sizes
    for i in range(len(state)):
        for j in range(i + 1, len(state)):
            if state[i] + state[j] <= cap:
                rest = list(state);  merged = rest[i] + rest[j]
```

```
                del rest[j];  del rest[i]
                rest.append(merged)
                if not wins(tuple(sorted(rest))): return True
    return False                          # no move, or every move loses

def best_guess(words, candidates):
    def worst_group(guess):
        groups = Counter(matches(guess, w) for w in candidates)
        return max(groups.values())
    return min(words, key=worst_group)
```

Cost: Number of distinct states times moves per state; guessing is O(words * candidates). **Traps:** Normalise the state (sort) so equal positions share one cache entry. Say who moves next and what "no move" means (a loss in normal play). Terminal states need a base case.

Follow-ups: Count winning first moves. Misere play. A sum of independent games (Sprague-Grundy values). Weighted answer likelihoods. How big is the real state space?

Solve these first: warm-up M, Choose optimal guesses for green-only Wordle; then stretch H, Babylon Tile-Stack Merge Game.

Questions in this section (E easy, M medium, H hard):

M **Choose optimal guesses for green-only Wordle**: bitmask per (position, letter) of the words that have it; the best guess splits the words into the most even groups
H **Determine winner in stack-merging game**: state = the sorted tuple of stack sizes; a move merges two whose sum stays within the cap
H **Find secret word with match-count feedback**: guess the word that minimises the size of the biggest feedback group; keep the words with the same match count
H **Wordle-Style 5-Letter Minimum-Guess Strategy**: ask about the letter at a position that splits the remaining words most evenly (min of yes-count and no-count is ...
H **Babylon Tile-Stack Merge Game**: state = the sorted tuple of stacks; a move merges two stacks that share a height or a colour; you win if some merge ...

19. Rolling state: the next state comes only from the previous one

Dynamic programming and memoization 7 questions: 2 easy, 4 medium, 1 hard

Spot it: "climbing stairs", "house robber", "paint houses", "fibonacci", "dice with limits".

Idea: If dp[i] depends only on dp[i-1] (and dp[i-2]), keep just those few variables instead of the full table. Climbing stairs: (a, b) = (b, a + b). House robber: keep (best if you take this house, best if you skip it). Paint house with k colours: keep a row of costs and, for each colour, add the smallest previous value that is not the same colour (store the two smallest).

```
def rob(money):
    take, skip = 0, 0
    for x in money:
        take, skip = skip + x, max(take, skip)
    return max(take, skip)

def climb(n):
    a, b = 1, 1
    for _ in range(n - 1):
        a, b = b, a + b
    return b
```

Cost: O(n) time, O(1) space (O(k) for a row of colours). **Traps:** Initialise with the true base cases. Update both variables in one tuple assignment. Circular houses: run twice, once without the first house and once without the last.

Follow-ups: n up to 10^18 (matrix exponentiation, or fast doubling). Steps 1 to k (sliding window sum). Broken steps. Return which houses (keep parent choices). Closed form (Binet) and its floating point limits.

Solve these first: warm-up E, Climbing Stairs; then stretch H, Constrained Dice Roll Sequences.

Questions in this section (E easy, M medium, H hard):

E **Climbing Stairs**: state = the last two counts: a, b = b, a + b, repeated n - 1 times
E **Fibonacci Numbers**: the same pair update starting from (0, 1)
M **Maximize sum without choosing adjacent elements**: exactly the house robber update on the array
M **Paint House II**: state = a whole row of costs, one per colour; new[j] = cost[j] + the smallest previous value that is not colour j (keep ...
M **Given n houses in a line with money in each, find the maximum ...**: state = (best if you take this house, best if you skip it): take, skip = skip + x, max(take, skip)
M **Write SQL code to publish the Fibonacci series.**: WITH RECURSIVE keeps (n, a, b); every new row is (n + 1, b, a + b): the same rolling pair, written as a query
H **Constrained Dice Roll Sequences**: state = a table [value][run length]; a different value starts a run of 1, the same value extends its run (up to the ...

5. Recurrence on a position or a range (memo or table)

Dynamic programming and memoization 9 questions: 0 easy, 6 medium, 3 hard

Spot it: "edit distance", "longest palindromic subsequence", "word break", "split into expressions", "count decompositions".

Idea: Define best(i) for a suffix or best(i, j) for a range in words, write how it follows from smaller states, and either memoise the recursion or fill a table in an order where smaller states come first. Strings: dp[i][j] for prefixes of two strings (equal letters copy the diagonal, otherwise 1 + the cheapest neighbour). Ranges: best(l, r) tries every split point m. Digit DP walks the number from the top with a tight flag and a state summary.

```
def edit_distance(a, b):
```

```
    previous = list(range(len(b) + 1))
    for i in range(1, len(a) + 1):
        current = [i] + [0] * len(b)
        for j in range(1, len(b) + 1):
            if a[i - 1] == b[j - 1]: current[j] = previous[j - 1]
            else: current[j] = 1 + min(previous[j], current[j - 1], previous[j - 1])
        previous = current
    return previous[-1]

@lru_cache(maxsize=None)
def ways(i):                              # decompositions of text[i:]
    if i == len(text): return 1
    return sum(ways(i + len(w)) for w in dictionary if text.startswith(w, i))
```

Cost: States times transitions, for example O(n * m) for edit distance and O(n^2 * split) for range DP. **Traps:** Write the state and recurrence in plain words before coding. Base cases. For palindromic subsequences the table is indexed by (i, j) with i <= j. Memoised recursion can hit the recursion limit on long inputs.

Follow-ups: Return the actual solution (keep choices). Count modulo a prime. Count in a range [A, B] (count up to B minus up to A-1). Faster version (Fenwick tree, sparse table). Different costs per operation.

Solve these first: warm-up M, Target Expressions from Ordered Digits; then stretch H, Edit distance.

Questions in this section (E easy, M medium, H hard):

- M **Target Expressions from Ordered Digits**: best(l, r) = every value reachable from digits l..r; split at m and combine the two sides with +, - and *
- M **Count Word Decompositions into Chemical Element Symbols**: bottom-up ways[i]: add ways[i+1] if the one-letter symbol exists and ways[i+2] if the two-letter symbol exists
- M **Fence Painting Minimum Operations (Gemini Team)**: range (l, r): paint one stripe at the lowest height, then solve the pieces on both sides; a sparse table finds the ...
- M **Maximize 1D deliveries within distance and minimize total ...**: table over (orders used, drivers used): skip an order, skip a driver, or match them when they are within D (count, then ...
- M **Find the longest palindromic subsequence using dynamic ...**: table over (i, j): equal ends add 2 to the inside; otherwise the better of dropping either end
- M **Jump Game with Profit**: best[i] = max over every j before i that can reach i (j + arr[j] >= i) of best[j] + profit[i]
- H **Count Perfect Wake Numbers up to N**: digit DP: cache best(remaining digits, used-digit mask, last digit, falling?) and walk along N with a tight flag
- H **Word Break: All Sentences**: best(start) = the list of sentences for the suffix; for every word that matches at start, put it in front of each ...
- H **Edit distance**: table over two strings, two rows: equal letters copy the diagonal, otherwise 1 + the smallest of the three neighbours

26. Grid DP: each cell from its neighbours above and to the left

Dynamic programming and memoization 5 questions: 0 easy, 3 medium, 2 hard

Spot it: "unique paths", "largest square", "longest run of one colour", "paths with checkpoints".

Idea: Fill the table row by row; each cell depends on cells already filled. Number of paths: ways[r][c] = ways[r-1][c] + ways[r][c-1] (zero on blocked cells or missed checkpoints). Same-colour squares: if the cell and its three neighbours match, side[r][c] = 1 + min of the three neighbouring sides. Longest run in four directions: keep four run lengths per cell from the previous row and the left cell.

```
squares = 0
side = [[0] * cols for _ in range(rows)]
for r in range(rows):
    for c in range(cols):
        if r and c and grid[r][c] == grid[r-1][c] == grid[r][c-1] == grid[r-1][c-1]:
            side[r][c] = 1 + min(side[r-1][c], side[r][c-1], side[r-1][c-1])
        else:
            side[r][c] = 1
        squares += side[r][c]               # a square of side s counts s squares
```

Cost: O(rows * cols) time, O(cols) space if you keep only the previous row. **Traps:** Edges (first row and column). Diagonal moves need three predecessors. A checkpoint resets rows that do not pass through it.

Follow-ups: Largest square only. Allow one different cell. Grid arrives row by row (keep one row). Return where the run is.

Solve these first: warm-up M, Grid Path Count with Diagonal Moves and Checkpoints; then stretch H, Count same-color squares in a character grid.

Questions in this section (E easy, M medium, H hard):

- M **Grid Path Count with Diagonal Moves and Checkpoints**: advance column by column keeping the number of ways in each row; new[r] = old[r-1] + old[r] + old[r+1]; zero out rows ...
- M **Longest Same-Color Grid Sequence**: keep four run lengths per cell (horizontal, vertical, two diagonals) from the previous row and the left cell; the ...
- M **Count Same-Color Squares in Matrix**: the pattern template above: a square ends here if the three neighbours match; its size is 1 + min of their sizes
- H **Find the Longest Straight Run of One Color**: the same four run lengths; extend a run when the neighbour in that direction has the same colour
- H **Count same-color squares in a character grid**: exactly the same recurrence on characters

32. Counting or reaching an amount: the knapsack loop

Dynamic programming and memoization 5 questions: 0 easy, 5 medium, 0 hard

Spot it: "coin change", "partition equal subset sum", "target sum with plus and minus", "unbounded items that fill a capacity".

Idea: dp[t] is the answer for total t. For each item, update dp across totals. Use items unlimited times: loop t upward (dp[t] = min(dp[t], dp[t - c] + 1)). Use each item once: loop t downward so an item cannot be used twice in one pass. For plus/minus signs, rewrite as a subset sum of (total + target) / 2. A bitset (Python int) makes subset sum fast: reach |= reach << x.

```
def coin_change(coins, amount):
    best = [0] + [float('inf')] * amount
    for t in range(1, amount + 1):
        for c in coins:
            if c <= t: best[t] = min(best[t], best[t - c] + 1)
    return -1 if best[amount] == float('inf') else best[amount]

def can_partition(nums):
    total = sum(nums)
    if total % 2: return False
    reach = 1                               # bit t is set if total t is reachable
    for x in nums: reach |= reach << x
    return (reach >> (total // 2)) & 1 == 1
```

Cost: O(amount * number of coins), or O(n * sum) for subset sum. **Traps:** Count combinations (loop items outside) versus permutations (loop totals outside). Direction of the inner loop decides reuse. Negative numbers break the plain table.

Follow-ups: Count the ways (LeetCode 518). Return which coins. Minimise the difference of the two parts. Does greedy work (only for special coin systems)?

Solve these first: warm-up M, Maximize Shelf Space (Unbounded Knapsack); then stretch M, Reverse Coin Change.

Questions in this section (E easy, M medium, H hard):

- M **Maximize Shelf Space (Unbounded Knapsack)**: reachable[t] = any(reachable[t - size]); the answer is the largest reachable total up to the capacity
- M **Coin Change**: fewest coins: start with infinity, best[t] = min(best[t], best[t - c] + 1); -1 when it stays infinity
- M **Partition an array into two sub-arrays with equal sum.**: subset sum to total / 2, written with a bitset: reach |= reach << x
- M **Explain how to find a target sum in an array.**: turn +/- signs into a subset sum of (total + S) / 2, items used once (sweep downwards)
- M **Reverse Coin Change**: given the ways table, find the coin values: the first amount whose count is wrong must be a coin; add it and rebuild

4. A heap of what is available: pop the best, put back the changed one

Heaps and top-k 9 questions: 0 easy, 8 medium, 1 hard

Spot it: "minimum rooms", "streaming median", "reusable locker", "schedule on identical CPUs", "no two adjacent the same".

Idea: Keep a heap of the things you can pick right now, keyed by what makes one better. Pop the best, use it, and push it back with its new state (a room with a new end time, a task with a smaller count). Two heaps hold a stream median: a max-heap for the lower half, a min-heap for the upper half, rebalanced so sizes differ by at most 1. Python only has a min-heap, so negate values for a max-heap.

```
free_at = []                                # heap of end times of rooms in use
for start, end in sorted(meetings):
    if free_at and free_at[0] <= start:
        heapq.heapreplace(free_at, end)     # reuse the room that frees first
    else:
        heapq.heappush(free_at, end)        # open a new room
rooms = len(free_at)

low, high = [], []                          # max-heap (negated), min-heap
def add(x):
    heapq.heappush(low, -x)
    heapq.heappush(high, -heapq.heappop(low))
    if len(high) > len(low): heapq.heappush(low, -heapq.heappop(high))
def median(): return -low[0] if len(low) > len(high) else (-low[0] + high[0]) / 2
```

Cost: O(log n) per push and pop. **Traps:** Tuples compare left to right: add a tie-breaker (an id) so unorderable objects never compare. Lazy deletion for stale entries. State why greedy is right (the soonest-free resource is the best one to reuse).

Follow-ups: Return which room each meeting uses. Rank by busy time. Online arrivals. Sliding median (two heaps with lazy deletion). Songs by the same artist at least k apart (hold artists in a cooldown queue).

Solve these first: warm-up M, Streaming Median (LC 295) with Follow-ups; then stretch H, Huffman Tree Construction.

Questions in this section (E easy, M medium, H hard):

- M **Streaming Median (LC 295) with Follow-ups**: two heaps: a max-heap for the lower half and a min-heap for the upper half; keep them balanced
- M **Compute minimum number of rooms needed**: heap of end times; a meeting reuses the room that frees first if it ended, otherwise push a new room
- M **Implement a Reusable Locker Allocation System**: heap of freed locker numbers; take the smallest freed one before handing out a new number
- M **Simulate room assignments for scheduled meetings**: a heap of free room numbers and a heap of (end time, room); a delayed meeting takes the earliest-ending room and shifts ...
- M **Minimum Makespan on Identical CPUs**: the pattern template above: heap of the times the CPUs become free
- M **Design server allocation for multi-type nodes**: a heap per type of (flexibility, server); skip servers already allocated (lazy deletion)
- M **Trip Expense Settlement**: a heap of creditors and a heap of debtors; the biggest of each settles min(amount) and the remainder goes back
- M **Playlist Shuffle No Adjacent Artists**: pop the artist with the most songs left, hold it for one turn, then push it back with a smaller count
- H **Huffman Tree Construction**: pop the two smallest weights, push their sum as a new node; repeat until one node remains

35. K-way merge: a heap of the next item from each source

Heaps and top-k 5 questions: 1 easy, 2 medium, 2 hard

Spot it: "merge k sorted lists", "k smallest pair sums", "sort a nearly sorted array", "top-k subset sums".

Idea: The smallest unseen item overall must be the head of one of the sources. Keep the current head of each source in a heap, pop the smallest, and push its successor (the next item of the same source). For pair sums of two sorted arrays, seed with (a[i] + b[0], i, 0); the successor of (i, j) is (i, j+1). For a k-sorted array, keep a heap of k + 1 items and always emit the smallest.

```
def merge_k(lists):
    heap = [(lst[0], i, 0) for i, lst in enumerate(lists) if lst]
    heapq.heapify(heap)
    out = []
    while heap:
        value, which, pos = heapq.heappop(heap)
        out.append(value)
        if pos + 1 < len(lists[which]):
            heapq.heappush(heap, (lists[which][pos + 1], which, pos + 1))
    return out
```

Cost: O(N log k) for N items in total from k sources. **Traps:** Include the source id so equal values never compare lists or nodes. Seed from the largest end if you want the largest items. Do not merge lists one by one into a growing result (O(N k)).

Follow-ups: Lists too large for memory (streams or files, same algorithm). Not sorted (sort first or use a selection algorithm). Three arrays. Return the subsets themselves.

Solve these first: warm-up E, Find k pairs with smallest sums; then stretch H, Merge k sorted linked lists..

Questions in this section (E easy, M medium, H hard):

- E **Find k pairs with smallest sums**: seed with (a[i] + b[0], i, 0); the successor of pair (i, j) is (i, j + 1)
- M **K Largest Elements Across Sorted Lists**: seed with the LAST item of each list (negated); the successor is the item before it; reverse the result
- M **Sort a nearly sorted (or K sorted) array.**: keep a heap of the first k + 1 items; pop the smallest and push the next input item (heapreplace)
- H **Return top-k subset sums in descending order**: heap of (sum, index); each popped state pushes two successors: add the next item, or swap the last item for the next
- H **Merge k sorted linked lists.**: seed with each list head (value, list number, node); the successor of a node is node.next

23. Top-K by frequency: Counter plus nsmallest

Heaps and top-k 6 questions: 0 easy, 6 medium, 0 hard

Spot it: "k most frequent words", "top chatters", "task scheduler with cooldown", "bottom k".

Idea: Count with Counter, then take the k best with a size-k heap: heapq.nsmallest(k, items, key=lambda w: (-count[w], w)) gives most frequent first with ties by name. This costs O(d log k) for d distinct items instead of sorting everything. For a cooldown scheduler, only the biggest count matters: answer = max(len(tasks), (biggest - 1) * (n + 1) + number of tasks tied for the biggest).

```
counts = Counter(word for line in lines for word in line.split()[:1])
top = heapq.nsmallest(k, counts, key=lambda w: (-counts[w], w))

def least_interval(tasks, n):
    counts = Counter(tasks).values()
    biggest = max(counts)
    tied = sum(1 for c in counts if c == biggest)
    return max(len(tasks), (biggest - 1) * (n + 1) + tied)
```

Cost: O(N + d log k) time, O(d) space. **Traps:** State the tie-break rule. A stream with too many distinct keys needs an approximate counter (Count-Min sketch, Misra-Gries). The log may not fit on one machine: count per machine and merge, or partition by hash.

Follow-ups: Unbounded stream and bounded memory. top_k must be O(k) per call (maintain buckets by count). Different cooldowns or priorities. Print an actual schedule (max-heap of counts plus cooldown queue).

Solve these first: warm-up M, Top-K Frequent in Large Logs; then stretch M, Top-K Chatters by Word Count.

Questions in this section (E easy, M medium, H hard):

- M **Top-K Frequent in Large Logs**: count the first word of every line (the speaker); return the names of the k most frequent
- M **Rearrange Tasks With Cooldown**: only the biggest count matters: answer = max(len(tasks), (biggest - 1) * (n + 1) + how many tasks share the biggest ...
- M **Implement frequency + distance top-K queries**: count the point tuples; the tie-break is distance from the origin, then x, then y
- M **Find top/bottom-k words in list or stream**: the same Counter; run nsmallest once with -count and once with +count for the bottom-k
- M **Solve chat, grid paths, and car rentals**: the top-K part is the pattern template above; the other parts are a rolling-column path count and a greedy car ...
- M **Top-K Chatters by Word Count**: count len(body.split()) per speaker instead of lines; ties by name

1. Frontier search (BFS or DFS), tier 1: spread, count, measure distance

Graph traversal: BFS and DFS 9 questions: 1 easy, 6 medium, 2 hard

Spot it: "number of islands", "flood fill", "rotten oranges", "shortest path in an unweighted graph", "distance to nearest X".

Idea: A frontier search has starts, a rule for which neighbours are usable, and a seen set. DFS or BFS both visit everything reachable. Use BFS when you need distances (layer by layer, each layer is one more step). Multi-source BFS puts every start in the queue at

distance 0, so you get the distance to the nearest start for all cells in one pass. Counting components: loop over all cells and start a search from each unseen land cell.

```
def nearest_distance(grid, starts):
    rows, cols = len(grid), len(grid[0])
    dist = {s: 0 for s in starts}
    queue = deque(starts)
    while queue:
        r, c = queue.popleft()
        for nr, nc in ((r+1, c), (r-1, c), (r, c+1), (r, c-1)):
            if 0 <= nr < rows and 0 <= nc < cols and (nr, nc) not in dist \
                    and grid[nr][nc] != '#':
                dist[(nr, nc)] = dist[(r, c)] + 1
                queue.append((nr, nc))
    return dist                             # missing key = unreachable
```

Cost: O(V + E), for a grid O(rows * cols). **Traps:** Mark cells seen when you enqueue, not when you dequeue (otherwise duplicates). Unreached cells stay unreachable (-1). To rebuild a path store parent[v] = u at first discovery.

Follow-ups: Huge grid (stream rows, or union-find). Cells change over time (rebuild only the affected area). Return every shortest path or count them (add counts per layer). Weighted edges (Dijkstra).

Solve these first: warm-up E, Flood Fill; then stretch H, Shortest Distance From All Buildings.

Questions in this section (E easy, M medium, H hard):

- E **Flood Fill**: starts = [(sr, sc)]; usable = same colour as the start; repaint every cell you reach; return early if old colour == new ...
- M **Number of Islands**: loop over every cell; each unseen land cell starts a new frontier_search (share one seen set); islands += 1 per start
- M **Rotten Oranges** / **Multi-Source BFS (taxis)**: starts = every rotten orange; usable = fresh orange; answer = rounds taken, or -1 if a fresh orange stays unreached
- M **Find shortest relationship path using BFS**: one start; also store parent[v] = u when you first reach v; stop at the target and walk parent back to rebuild the path
- M **Compute distance to nearest taxi in grid**: starts = every taxi cell; usable = not a wall; the dist table is the answer, cells never reached stay -1
- M **Solve Grid and Counting Problems**: the island count again (queue version); the second part of the question is a short counting formula, no search
- M **Farthest City for a New Data Center**: same on a graph: starts = the data centres, neighbors = adjacency list; answer = the city with the largest distance
- H **Infection Spread on a Grid (Cellular Automaton)**: starts = every infected cell; usable = healthy cell; the answer is the largest distance reached (number of rounds)
- H **Shortest Distance From All Buildings**: run it once per building (one start each); add each run into a total table and count how many buildings reached each ...

28. Frontier search, tier 2: free steps, early exit, neighbours worked out on the fly

Graph traversal: BFS and DFS 5 questions: 0 easy, 5 medium, 0 hard

Spot it: "edge direction flips cost 1", "reach with blocked nodes", "shortest directed cycle through v", "radios within a radius".

Idea: Same BFS, but the graph is not given as lists: neighbours(u) is computed when needed (all radios within a radius, using squared distances). A mix of free and paid moves is 0-1 BFS: a deque where free steps use appendleft and paid steps use append, so the deque stays sorted by distance. Early exit: stop when the goal is popped. Shortest cycle through v: start at v and return dist[u] + 1 the moment a neighbour of u equals v.

```
def zero_one_bfs(graph, source):
    dist = {source: 0};  queue = deque([source])
    while queue:
        u = queue.popleft()
        for v, weight in graph[u]:          # weight is 0 or 1
            if dist[u] + weight < dist.get(v, float('inf')):
                dist[v] = dist[u] + weight
                if weight == 0: queue.appendleft(v)
                else:           queue.append(v)
    return dist
```

Cost: O(V + E) for BFS and 0-1 BFS; O(V^2) when neighbours are computed from all other nodes. **Traps:** Compare squared distances to avoid square roots. A blocked source means everything is unreachable. A node can be improved after first discovery in 0-1 BFS, so check strictly smaller.

Follow-ups: Arbitrary non-negative weights (Dijkstra). Shortest cycle in the whole graph (BFS from every node). Nodes blocked over time. Queries on the same graph (precompute adjacency).

Solve these first: warm-up M, Shortest Path Nodes and Escaping a Pursuer; then stretch M, Shortest Directed Cycle Through a Node.

Questions in this section (E easy, M medium, H hard):

- M **Shortest Path Nodes and Escaping a Pursuer**: run it from a, from t and from b; a node is on a shortest a-to-t path when da + dt == da[t]; compare with the pursuer's ...
- M **Radio Broadcast Reachability**: neighbors(u) = every radio within the radius, worked out with squared distances; goal = the target radio, answer = ...
- M **Minimum Direction Violations in a Directed Graph**: an edge in its own direction costs w = 0, the same edge walked backwards costs w = 1; free steps use appendleft
- M **Compute shortest paths with blocked nodes**: neighbors skip every blocked node; if the source itself is blocked the answer is all -1; unreachable nodes stay -1
- M **Shortest Directed Cycle Through a Node**: start = v; the goal is an edge that leads back to v, so return dist[u] + 1 the moment a neighbour equals v

29. Topological sort (Kahn) and cycle detection

Topological sort and cycles 5 questions: 0 easy, 5 medium, 0 hard

Spot it: "course schedule", "build order", "are these race results consistent", "validate dependencies".

Idea: Compute the in-degree of every node. Put all in-degree 0 nodes in a queue, repeatedly remove one, append it to the order, and decrease its neighbours' in-degrees (adding new zeros). If the order contains every node the graph has an order; otherwise the remaining nodes lie on or behind a cycle. Layering by rounds gives the minimum number of semesters.

```
def topo_order(nodes, edges):              # edge (a, b): a must come before b
    indegree = {v: 0 for v in nodes};  after = defaultdict(list)
    for a, b in edges:
        after[a].append(b);  indegree[b] += 1
    queue = deque(v for v in nodes if indegree[v] == 0)
    order = []
    while queue:
        v = queue.popleft();  order.append(v)
        for w in after[v]:
            indegree[w] -= 1
            if indegree[w] == 0: queue.append(w)
    return order if len(order) == len(indegree) else None
```

Cost: O(V + E). **Traps:** Nodes that only appear in edges still need an in-degree entry. Direction of the edge (prerequisite to course). Duplicate edges inflate the in-degree consistently, which is fine.

Follow-ups: Report one actual cycle (DFS with colours). All missing references. Semesters (process the queue layer by layer). Recursive DFS overflow on long chains (use Kahn).

Solve these first: warm-up M, Course Scheduling and Dependency Variants; then stretch M, Course Schedule.

Questions in this section (E easy, M medium, H hard):

- M **Course Scheduling and Dependency Variants**: build the vertex set from the edges too; return the order or None
- M **Validate Race Results (Cycle Detection)**: edge = winner -> loser; the results are consistent exactly when there is no cycle
- M **Solve tree, graph, sliding-window problems**: the course-schedule part is can-finish: len(order) == n; the zigzag and window parts are separate small loops
- M **Validate course catalog dependencies**: first reject prerequisites naming courses that do not exist; then the same peel; report the missing names or the cycle
- M **Course Schedule**: edge = prerequisite -> course; return the order, or an empty list when there is a cycle

9. Union-find: merge the pairs, then ask who is together

Union-find and connectivity 8 questions: 0 easy, 7 medium, 1 hard

Spot it: "connected components", "groups", "are A and B related", "add land one cell at a time", "points within a radius".

Idea: Each element points to a parent; the root is the group name. find(x) follows parents (with path compression), union(a, b) links the two roots (by size). Pairs may come from edges, from neighbouring land cells, or from points within a radius (bucket points into cells of size r and test only the nine nearby cells). Offline path problems: sort edges from best to worst and union until two nodes meet; the last edge is the threshold.

```
parent = {}
def find(x):
    parent.setdefault(x, x)
    while parent[x] != x:
        parent[x] = parent[parent[x]]      # path halving
        x = parent[x]
    return x
def union(a, b):
    ra, rb = find(a), find(b)
    if ra == rb: return False              # already together
    parent[ra] = rb;  return True

groups = len(nodes)
for a, b in pairs:
    if union(a, b): groups -= 1
```

Cost: Nearly O(1) per operation (inverse Ackermann) with path compression and union by size. **Traps:** Unfriending breaks plain union-find (use offline reverse processing or rebuild). Count groups by decrementing only on a real merge. Keep extra data (boss, size) at the root and carry it over when merging.

Follow-ups: Land can be removed. Many (source, target) queries (build once). Report the largest group size (track size at the root). Directed reachability is not union-find (use BFS or topological order).

Solve these first: warm-up M, Count Islands in a Grid and Track the Count as Land Cells Are Added; then stretch H, Friend Graph with Timeline (Friend / Unfriend Events).

Questions in this section (E easy, M medium, H hard):

- M **Count Islands in a Grid and Track the Count as Land Cells Are ...**: each new land cell: groups += 1, then union it with every land neighbour (groups -= 1 per real merge); report groups ...
- M **2D Points Connected Components by Radius**: pairs = points within distance r, found by bucketing points into cells of size r and checking only the 9 nearby cells ...

- M **Count Clusters of 2D Points Within a Radius**: identical to the line above: cell buckets give the candidate pairs, answer = groups
- M **Find A Threshold-Limited Path With Minimum Required Safety**: pairs = edges sorted from the highest safety down; union until find(s) == find(t); the answer is the safety of that ...
- M **Determine Reporting Relationships**: pairs = peer links; keep the boss at the root and carry it over when two groups merge
- M **Determine whether two nodes are related**: pairs = the given edges; the query is find(a) == find(b)
- M **Movie Recommendations via Transitive Similarity**: pairs = similar movies (use a dict instead of a list); group = every movie with the same root as the target, sorted by ...
- H **Friend Graph with Timeline (Friend / Unfriend Events)**: pairs = friend events sorted by time; the answer is the time of the union that makes groups == 1

6. Best-first search with a heap: Dijkstra and its minimax twin

Dijkstra and shortest paths 9 questions: 0 easy, 5 medium, 4 hard

Spot it: "shortest path with weights", "earliest arrival with timetables", "minimise the maximum height on a path", "swim in rising water".

Idea: Always expand the unfinished node with the smallest cost so far, kept in a heap. When a node is popped its cost is final (weights non-negative). The cost does not have to be a sum: use max(cost, edge) for "minimise the worst step" (minimax), or the arrival time at the next station for timetables (you may leave only at or after you arrive). For maximising a quantity, push negated values.

```
def dijkstra(graph, source, combine=lambda cost, w: cost + w):
    best = {source: 0};  heap = [(0, source)]
    while heap:
        cost, u = heapq.heappop(heap)
        if cost > best.get(u, float('inf')): continue    # stale entry
        for v, w in graph[u]:
            new_cost = combine(cost, w)
            if new_cost < best.get(v, float('inf')):
                best[v] = new_cost
                heapq.heappush(heap, (new_cost, v))
    return best

# minimax: combine=lambda cost, w: max(cost, w)
```

Cost: O((V + E) log V). **Traps:** Skip stale heap entries. Negative edges break it (use Bellman-Ford). Track parent[v] to rebuild the path. With unit weights plain BFS is enough; with only 0 and 1 use 0-1 BFS.

Follow-ups: Return the path. Fewest transfers. Several travellers (run from each, combine at meeting nodes). Weights change over time. Layover rules (shift the departure test).

Solve these first: warm-up M, Path Minimizing the Maximum Cell (Swim in Rising Water); then stretch H, Compute minimax grid path and network delay.

Questions in this section (E easy, M medium, H hard):

- M **Path Minimizing the Maximum Cell (Swim in Rising Water)**: the same minimax loop with a given start and end cell (Swim in Rising Water)
- M **Nearest Favorite City by Travel Time**: combine = cost + w; stop when a favourite is popped (keep popping equal distances to break ties on the smaller index)
- M **Given a train schedule with departure city, arrival city ...**: cost = arrival time; neighbors only offers trains that leave at or after the current time; combine = the train's ...
- M **Find the shortest path in a graph.**: combine = cost + w; also store parent[v]; stop at the target and rebuild the path
- M **Given a matrix with a rat, bread, cat, and obstacles, find a ...**: maximise the minimum distance to the cat along the path: push negated values, combine = min(value, cat distance of the ...
- H **Minimize maximum height along a grid path**: the same minimax loop on a height grid
- H **Find minimum threshold enabling grid path**: the same minimax loop: the smallest threshold that lets you walk from corner to corner
- H **Minimize travel time with optional meeting point**: three runs: from a, from b, and from the destination on the reversed graph; answer = min over meeting points m of ...
- H **Compute minimax grid path and network delay**: combine = max(cost, height of the next cell); the answer is the cost at the bottom-right cell

Appendix: algorithms hiding inside single questions

These show up once or twice (often as a follow-up) but are worth knowing by name.

Fenwick tree (binary indexed tree)

Prefix sums with point updates in $O(\log n)$. Used for count of smaller numbers after self, dynamic range sums, and "array changes between queries" follow-ups. Index from 1; the lowest set bit of an index says how many items its cell covers.

```
class Fenwick:
    def __init__(self, n):  self.tree = [0] * (n + 1)
    def add(self, i, delta):                # i is 1-based
        while i < len(self.tree):
            self.tree[i] += delta;  i += i & -i
    def prefix(self, i):                    # sum of items 1..i
        total = 0
        while i > 0:
            total += self.tree[i];  i -= i & -i
        return total
# count of smaller numbers after self: go from the right, answer = prefix(rank - 1), then add(rank, 1)
```

Segment tree (first slot that fits, range minimum, range updates)

Store a summary of each range in a tree over positions. The waitlist question keeps the maximum party size per range, so "first group of at most s" descends left when the left half has a big enough slot. Cost $O(\log n)$ per update and query.

```
class MaxTree:
    def __init__(self, values):
        self.n = 1
        while self.n < len(values): self.n *= 2
        self.tree = [0] * (2 * self.n)
        for i, v in enumerate(values): self.tree[self.n + i] = v
        for i in range(self.n - 1, 0, -1):
            self.tree[i] = max(self.tree[2 * i], self.tree[2 * i + 1])
    def update(self, i, value):
        i += self.n;  self.tree[i] = value
        while i > 1:
            i //= 2;  self.tree[i] = max(self.tree[2 * i], self.tree[2 * i + 1])
    def first_at_least(self, need):         # leftmost index with value >= need
        if self.tree[1] < need: return -1
        i = 1
        while i < self.n:
            i = 2 * i if self.tree[2 * i] >= need else 2 * i + 1
        return i - self.n
```

Rolling hash (Rabin-Karp) for repeated substrings

Longest duplicated substring: binary search the length L (pattern 5), and for each L slide a hash over the string in $O(n)$, storing hashes in a set. Roll by removing the leading character and adding the next. Verify a hash hit to rule out collisions, or use two moduli.

```
def has_duplicate(s, length, base=131, mod=(1 << 61) - 1):
    if length == 0: return True
    power = pow(base, length - 1, mod);  h = 0;  seen = set()
    for i, ch in enumerate(s):
        if i >= length: h = (h - ord(s[i - length]) * power) % mod
        h = (h * base + ord(ch)) % mod
        if i >= length - 1:
            if h in seen: return True
            seen.add(h)
    return False
```

Sparse table (range minimum in $O(1)$)

Fence painting asks for the lowest stripe in a range again and again. Precompute `table[j][i]` = minimum of the 2^j items starting at i; a query takes two overlapping blocks.

```
def build(a):
    table = [a[:]];  j = 1
    while (1 << j) <= len(a):
        prev = table[-1]
        table.append([min(prev[i], prev[i + (1 << (j - 1))])
                      for i in range(len(a) - (1 << j) + 1)])
        j += 1
    return table
def range_min(table, l, r):                 # inclusive
    j = (r - l + 1).bit_length() - 1
```

```
    return min(table[j][l], table[j][r - (1 << j) + 1])
```

Huffman coding

Greedy with a heap (pattern 24): repeatedly pop the two lightest nodes and push a parent whose weight is their sum. Reading the final tree, going left is 0 and going right is 1. Canonical codes assign codes by length then symbol so only the lengths need storing.

```
def huffman_lengths(weights):               # weights: {symbol: weight}
    heap = [(w, i, {s: 0}) for i, (s, w) in enumerate(weights.items())]
    heapq.heapify(heap);  tiebreak = len(heap)
    while len(heap) > 1:
        w1, _, left = heapq.heappop(heap);  w2, _, right = heapq.heappop(heap)
        merged = {s: d + 1 for s, d in {**left, **right}.items()}
        heapq.heappush(heap, (w1 + w2, tiebreak, merged));  tiebreak += 1
    return heap[0][2]                       # symbol -> code length
```

Fast doubling and matrix power (huge n recurrences)

When n is 10^{18}, Fibonacci-like recurrences are computed in $O(\log n)$ by squaring. Fast doubling: $F(2k) = F(k)(2F(k+1)-F(k))$ and $F(2k+1) = F(k)^2 + F(k+1)^2$. Apply the modulus after every multiplication.

```
def fib(n, mod=10**9 + 7):                  # returns (F(n), F(n+1))
    if n == 0: return (0, 1)
    a, b = fib(n // 2, mod)
    c = a * ((2 * b - a) % mod) % mod
    d = (a * a + b * b) % mod
    return (d, (c + d) % mod) if n % 2 else (c, d)
```

Group testing, augmenting paths and other named ideas

- **Group testing** (find all bad pairs with few oracle calls): test a group in one call; if it passes, discard the whole group; if it fails, split it. Lower bound is about $k \log(n/k)$ calls for k bad items.
- **Augmenting path** (match people to questions): try each preferred question; if taken, recursively ask the owner to move to another; undo on failure. This is bipartite matching in $O(V \cdot E)$.
- **Fisher-Yates** and **reservoir sampling**: shuffle in place from the end; to pick one random item from a stream, replace the current pick with the i-th item with probability $1/i$.
- **Count-Min sketch** / **Misra-Gries**: approximate top-k over an unbounded stream in fixed memory.
- **Sprague-Grundy**: a sum of independent games is won by the player who makes the xor of the game values non-zero (the follow-up behind “games on several piles”).
- **Morris traversal**: in-order walk of a BST in $O(1)$ extra space by temporarily threading predecessor links.